

Rebuilding Your Trading Application

Python, APIs, secure identity, testing,
Google Cloud and Kubernetes

A practical textbook and chapter-by-chapter workbook for Amol

Python

Model and calculate

APIs and identity

Validate and authorize

Durable state

Record and reconcile

Testing

Prove and challenge

Cloud and Kubernetes

Deploy and recover

Data and AI

Analyze without trading authority

Expanded edition 2 | 23 September 2026

60 chapters. Worked examples throughout. A paper-only reference lab and container manifests.

Learn with synthetic data and disposable infrastructure. No example authorizes live trading. Cloud resources can incur charges.

Contents

How to use this book	5
Part One Python From First Principles	8
Chapter 1 Programs values and execution	8
Chapter 2 Environments modules and a repeatable setup	10
Chapter 3 Conditions loops and collections	12
Chapter 4 Functions contracts and precise money	14
Chapter 5 Exceptions files time and logging	16
Chapter 6 Objects dataclasses and interfaces	18
Chapter 7 Iterators decorators typing and Python internals	20
Chapter 8 Algorithms complexity and practical data structures	22
Part Two APIs Databases and Authentication	24
Chapter 9 HTTP and API contracts	24
Chapter 10 FastAPI validation and dependency injection	26
Chapter 11 Relational database design	28
Chapter 12 Transactions concurrency and the outbox	30
Chapter 13 Redis as a cache queue and coordination tool	31
Chapter 14 Email ownership and secure login	32
Chapter 15 TOTP sessions and broker authorization	34
Part Three Trading State and Distributed Systems	35
Chapter 16 Model the lifecycle before writing the broker call	35
Chapter 17 Classic strategy math and exit behavior	37
Chapter 18 Configuration ownership and the full feature map	39
Chapter 19 Candles breakout watches and sector ranking	41
Chapter 20 Async concurrency and bounded work	43
Chapter 21 Broker adapters and WebSocket evidence	45

Chapter 22 Service boundaries recovery and architecture drawings	47
Part Four Frontend Integration and Software Testing	49
Chapter 23 Browser fundamentals for a backend developer	49
Chapter 24 Unit testing as executable design	51
Chapter 25 API and real integration tests	52
Chapter 26 Playwright with Python	53
Chapter 27 Fault injection performance and security tests	55
Chapter 28 CI and evidence based release decisions	57
Part Five Google Cloud and Production Operations	58
Chapter 29 Cloud fundamentals and resource hierarchy	58
Chapter 30 IAM service accounts and secrets	59
Chapter 31 Network design and a safe VM exercise	60
Chapter 32 Linux process supervision and deployment	62
Chapter 33 Nginx HTTPS and browser WebSockets	64
Chapter 34 Observability scheduling backups and cost	66
Chapter 35 Containers Terraform and managed runtime choices	68
Part Six Data Engineering Machine Learning and Certification	70
Chapter 36 Build an analytics path outside execution	70
Chapter 37 Statistics and honest backtesting	72
Chapter 38 Machine learning and MLOps	73
Chapter 39 Embeddings vector databases and retrieval	74
Chapter 40 Choose a certification path	76
Chapter 41 Professional specialization labs	78
Chapter 42 Original practice questions with explanations	80
Part Seven Your Rebuild Workbook	82
Chapter 43 The staged implementation contract	82
Chapter 44 Guided debugging cases and answer notes	85

Chapter 45 Senior developer interview preparation	87
Chapter 46 Completion rubric and daily practice	89
Part Eight Extended Google Cloud and Kubernetes Labs	90
Chapter 47 Networking from addresses to application requests	90
Chapter 48 Containerize the paper application	92
Chapter 49 What Kubernetes actually controls	94
Chapter 50 Your first local cluster with kind	95
Chapter 51 Deployments labels and a complete paper manifest	96
Chapter 52 Resource requests limits and health probes	99
Chapter 53 Services DNS and private browser access	100
Chapter 54 Configuration secrets RBAC and network policy	101
Chapter 55 Persistent storage and database placement	103
Chapter 56 Rollouts autoscaling jobs and safe ownership	104
Chapter 57 Deploy the paper image to GKE	105
Chapter 58 Workload identity Cloud SQL and delivery pipelines	107
Chapter 59 Infrastructure as code observability and cost control	108
Chapter 60 A full paper system capstone	110
Appendix A Glossary	112
Appendix B Reference reading order	113
Appendix C Complete paper lab source walkthrough	114

How to use this book

Prepared for Amol. Expanded edition 2, 23 September 2026.

This book teaches you to rebuild a trading application yourself, beginning with small Python programs and progressing to APIs, databases, secure login, asynchronous services, browser testing, cloud operations, and architecture. The aim is not to memorize your existing source code. It is to understand the decisions behind it, reproduce its useful behavior, and recognize where a different design would be safer.

Your existing application is the case study. You receive Chartink signals, resolve instruments, evaluate strategies, submit broker orders, reconcile actual fills, monitor exits, and show results in a browser. That one workflow contains nearly every difficulty that makes backend engineering interesting: untrusted input, timing, concurrency, partial failure, external contracts, persistent state, and human expectations.

This is a substantial project textbook and workbook, not an encyclopedia of every Python feature or a substitute for every professional cloud exam guide. This expanded edition adds a worked lesson to each of the original 46 chapters and 14 extended cloud and Kubernetes chapters. It gives you a deep common foundation and specialization labs. Completing it is evidence of learning, not a guarantee of employment, seniority, certification, trading profitability, or production safety.

How to use the book

Read in order the first time. For each chapter, explain the concept aloud, implement the exercise without opening the existing implementation, write tests, and only then compare designs. Keep a notebook with four headings: prediction, observed behavior, explanation, and remaining uncertainty. A test failure is useful when you can explain why it occurred.

The accompanying lab directory is a deliberately small reference implementation. It uses a simulated broker and local SQLite. It has no live-order adapter and is not a production deployment. Build your own version in a new repository; use the reference to compare behavior rather than to bypass the exercises. The later chapters specify larger systems you must implement yourself. They do not claim those systems already exist in the reference lab.

Code blocks fall into three categories. The lab's files are runnable together. Short Python examples demonstrate a concept and name their dependencies. Architecture sketches and pseudocode explain contracts, not complete deployable services. Cloud commands create billable resources only where explicitly stated. Run cloud exercises in a disposable project, never in a client's production project. Long code lines can wrap visually in the PDF; consult the editable source and bundled lab files for exact formatting. Install tools only in a separate learning environment.

Keep real access tokens, webhook secrets, client emails, encryption keys, Redis passwords, and trading records out of your learning repository. Use fictional symbols and sanitized fixtures. A secret pasted into a chat or committed to Git needs a rotation plan; deleting the visible text is not rotation. Do not casually rotate an encryption key without first planning how existing encrypted data will be decrypted and re-encrypted.

What exists and what you will add

The repository inspected for this book contains FastAPI, an HTML dashboard, Redis state and queues, Dhan and Kite adapters, a trade engine, authentication code, and separate API, alert, execution, market-feed, and reconciliation entry points. Its dependency file pins DhanHQ 2.2.0 and KiteConnect 5.0.1. Those are facts about this checkout, not a recommendation to use old pins forever.

PostgreSQL, SQLAlchemy, database migrations, a transactional outbox, BigQuery, Vertex AI, vector search, Terraform, and Kubernetes are learning extensions in this book. They are not silently attributed to the existing

application. Start with the tools that solve your current problem; adding every cloud product to a single trading path would increase cost and failure modes.

Existing reference	What to study there	What to question
app/main.py	API routes, auth, lifecycle, browser feed	How much responsibility belongs in one module?
app/trade_engine.py	Entry and exit decisions	Which rules should become pure functions?
app/dhan_broker.py	SDK boundary and feed recovery	Are responses validated before changing state?
app/kite_broker.py	A different broker contract	Are common names hiding different meanings?
app/redis_store.py	Persistence, key scope, TTL	Which facts must survive cache eviction?
app/market_feed_service.py	Subscription ownership and health	Connected, subscribed, or receiving fresh data?
app/execution_service.py	Queue consumption and worker lease	What happens between submission and acknowledgment?
app/reconciliation_service.py	Recovery and fallback pricing	What is broker evidence versus local expectation?
app/order_locks.py	Mutual exclusion and ownership	What happens when a lease expires during work?
app/auth.py, app/email_service.py	Email login workflow	Is verification atomic and abuse-resistant?
app/static/dashboard.html	Forms, fetch, WebSocket updates	Are browser labels backed by fresh evidence?
tests/	Unit, integration and browser checks	Which failure cannot this test actually detect?

Read code as evidence, not as scripture. A README can lag behind implementation; a test can encode a mistaken assumption. Follow a real input through the current routes and service boundaries.

Your development sequence

1. Build a command-line calculator for quantity, target, stop, and P&L.
2. Model immutable signals, orders, fills, positions, and strategy settings.
3. Add a deterministic paper broker and unit tests.
4. Save configuration and fills in a relational database.
5. Expose a validated HTTP API with a consistent error contract.
6. Add administrator bootstrap, password login, email verification, and TOTP.
7. Build a dashboard that displays server state and handles failures.
8. Add simulated ticks and confirm exits from actual simulated fills.
9. Add durable background jobs, duplicate protection, and recovery.
10. Implement broker adapters against sanitized fixtures and official contracts.
11. Split the measured bottlenecks into independently supervised processes.

12. Deploy a paper-only version to Google Cloud with HTTPS, monitoring, and restore tests.
13. Add analytics and ML as separate, non-executing extensions.
14. Conduct an architecture review, incident drill, and independent code review.

Do not begin with sixteen network services. A logical service can initially be a Python class. A process is an operating-system boundary; a distributed service adds network and operational boundaries. You can learn modularity before learning distributed failure.

A realistic study rhythm

Use four sessions per topic: learn and trace; implement; test and break; explain and refactor. At roughly ten focused hours each week, a 36-week first pass is a planning estimate, not a deadline. Someone new to programming may need longer. Advance when you can pass the chapter's exit exercise without following a tutorial.

Weeks 1-8 cover Python and algorithms. Weeks 9-14 cover API, SQL, and authentication. Weeks 15-21 cover trading state, feeds, and concurrency. Weeks 22-26 cover testing and browser integration. Weeks 27-32 cover cloud and operations. Weeks 33-36 cover one specialization and a capstone review. Work experience and maintaining software over time remain important for senior roles.

The essential safety rule

Use a simulator throughout this book. A successful HTTP response is not a confirmed fill. A green WebSocket badge is not proof of current prices. A passing test suite is not proof that every failure has been considered. Build evidence for each claim separately.

Sources and version policy

Official sources are linked near version-sensitive topics. They were consulted for this edition. Recheck SDK signatures, cloud product names, quotas, prices, exam guides, and broker requirements when you implement or book an exam. The cloud portfolio in your attachment is a list of possible paths, not a single syllabus, and its cross-vendor comparisons are approximate rather than equivalences.

Part One Python From First Principles

Chapter 1 Programs values and execution

A program is a sequence of instructions operating on values. Python's interpreter executes your source within a process. The process has memory, open files, sockets, and an exit status. A source file on disk is not a running process. Editing the file does not necessarily update a process that already imported it. This explains why a server sometimes needs a controlled restart after deployment.

A variable is a name bound to an object. The object has a type; the name can later refer to another object. `quantity = 5` binds an integer. `symbol = "RELIANCE"` binds text. A string containing "5" is not the integer 5. HTTP often carries text, so boundaries require conversion and validation.

PYTHON

```
symbol = "RELIANCE"
quantity = 5
entry = 100
latest = 103
pnl = (latest - entry) * quantity
print(f"{symbol}: quantity={quantity}, P&L={pnl}")
```

The output is `RELIANCE: quantity=5, P&L=15`. Change `latest` to 97 and predict the output before running. Change `quantity` to "5"; explain the result rather than guessing what Python should do. Multiplication of a string repeats it, demonstrating why syntactically legal code can be semantically wrong.

Learn these basic types: `int` for whole numbers, `float` for approximate real arithmetic, `bool` for truth values, `str` for Unicode text, `bytes` for encoded data, and `None` for absence. `0`, `False`, an empty list, and `None` are all false-like in conditions, but they do not mean the same thing.

One of your application's previous configuration risks illustrates this distinction:

PYTHON

```
payload = {"pending_retries": 0}
wrong = payload.get("pending_retries") or 1
raw = payload.get("pending_retries")
correct = 1 if raw is None else raw
assert wrong == 1
assert correct == 0
```

Zero is a legitimate request for no retries. Missing means the client did not specify a value. Never use truthiness as a substitute for a domain rule.

Build: Ask for a symbol, side, quantity, and price with `input()`. Convert quantity explicitly, reject nonpositive values, and print a paper-order summary. Do not connect to a broker.

Exit check: Explain source file versus process, text versus number, and missing versus zero. Include tests for blank input, negative quantity, and malformed price.

Worked lesson tracing a program by hand

Before typing, create a table with columns line, names, and output. Step through this example as the interpreter would:

PYTHON

```
price = 100
quantity = 2
notional = price * quantity
price = 105
print(notional)
print(price * quantity)
```

The outputs are 200 and 210. `notional` does not contain a spreadsheet formula that updates when price changes. It contains the result of an earlier multiplication. This matters when a dashboard displays an entry-time value instead of recalculating on ticks.

Now add `enabled = False` and an `if enabled:` branch. Indentation defines the suite controlled by the condition. Use four spaces consistently. A syntax error prevents interpretation; a runtime error occurs on an executed path; a logic error produces an incorrect result without an exception. You need different techniques for each: parser feedback, traceback inspection, and expected-behavior tests.

Independent exercise: Write three examples that produce the same printed output but use different types internally. Explain why printed output alone cannot validate a request schema. Then write an assertion about the type as well as the value.

Chapter 2 Environments modules and a repeatable setup

A virtual environment isolates Python packages. It does not isolate the operating system, network, or live broker account. A requirements file records dependencies; a lock or fully resolved constraints record makes installations more reproducible. Security updates still require review and tests.

Create a separate learning repository. In Windows PowerShell:

POWERSHELL

```
mkdir trading-school
cd trading-school
py -3.12 -m venv .venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
git init
```

On Ubuntu use `python3 -m venv .venv` and `.venv/bin/python`. Use Python 3.12 or later for this book; test SDK compatibility before upgrading production. Do not install teaching dependencies into your deployed application's virtual environment.

Use `python -m package.module` to execute a module with a predictable import context. A package organizes related modules. Imports execute top-level statements once per interpreter import cache, so avoid placing live broker calls, network downloads, or background thread startup at import time.

PYTHON

```
def main():
    print("Paper environment ready")

if __name__ == "__main__":
    main()
```

The guard prevents `main()` from running merely because another module imports the file. It is particularly important when process workers import modules during startup.

Your project should contain `src/` or one clear package, `tests/`, a dependency declaration, `.gitignore`, and a README with commands you have actually tested. Ignore `.env`, virtual environments, real datasets, and token-bearing screenshots. Commit a `.env.example` containing names and harmless placeholders only.

Build: Split your calculator into `money.py`, `risk.py`, and `cli.py`. Import functions rather than duplicating them. Explain what happens if two modules import each other. Resolve the cycle by moving shared types into a smaller module, not by hiding imports everywhere.

Debug drill: Deliberately install a package in the wrong interpreter. Compare `python -c "import sys; print(sys.executable)"` with the interpreter used by your service. Learn why "installed successfully" does not imply "available to this process."

Worked lesson package layout and interpreter identity

Create this layout in your own repository, not inside the production app:

TEXT

```
trading-school/
  school/
    __init__.py
    money.py
    cli.py
  tests/
    test_money.py
  pyproject.toml
  .gitignore
```

In `school/money.py`, define a function. In `school/cli.py`, import it with `from school.money import ...`. Run `python -m school.cli` from the repository root. Running a file from an arbitrary working directory can change which modules Python finds; avoid fixing import problems by scattering `sys.path.append` throughout application code.

Inspect the active environment without printing secrets:

BASH

```
python -c "import sys; print(sys.executable); print(sys.version)"
python -m pip --version
python -m pip show fastapi
python -m pip check
```

The interpreter path, pip path, installed distribution, and compatibility check answer different questions. Save exact dependency versions after a tested installation, review the result, and record the Python version as well. A virtual environment copied between operating systems is not a portable deployment artifact.

Exercise: Create a second environment without FastAPI. Predict which command fails in it. Repair the invocation rather than installing packages globally. Add a README command that works in a fresh checkout.

Chapter 3 Conditions loops and collections

Conditions choose a path. Loops apply a rule repeatedly. A list preserves order and permits duplicates. A set represents membership without duplicates. A dictionary maps keys to values. A tuple is an immutable sequence, useful for composite identities such as (exchange, security_id).

Never identify an instrument only by a display name if multiple exchanges can use the same symbol. Never identify a client's position only by symbol when strategy ownership matters.

PYTHON

```
raw_symbols = [" sbin ", "TCS", "SBIN", ""]
symbols = list(dict.fromkeys(
    item.strip().upper() for item in raw_symbols if item.strip()
))
assert symbols == ["SBIN", "TCS"]
```

This example assumes a list of strings. An API must validate that assumption first. `dict.fromkeys` removes duplicates while preserving first appearance. A set alone would lose that intended order.

Mutation changes an existing object. Assignment does not necessarily copy:

PYTHON

```
a = {"symbols": ["SBIN"]}
b = a.copy()
b["symbols"].append("TCS")
assert a["symbols"] == ["SBIN", "TCS"]
```

The dictionary copy is shallow; both dictionaries refer to the same inner list. Prefer immutable domain objects and deliberate construction rather than casually deep-copying huge state. Understand `copy.deepcopy`, but ask whether shared mutable state should exist at all.

Avoid modifying a dictionary while iterating through it. Gather keys to remove, then remove them, or construct a new dictionary. Use `enumerate` for positions and values, and `zip(..., strict=True)` when lengths must match. Silent truncation can hide mismatched quote and instrument arrays.

Build: Normalize a Chartink-like symbol list, retain input order, record rejected values, and produce one validation result per symbol. Add an explicit maximum count to prevent a huge payload from consuming unlimited memory.

Exit check: Explain why configuration dictionaries, subscription sets, and append-only fill lists have different purposes. Demonstrate aliasing with a ten-line program.

Worked lesson grouping without cross strategy leakage

PYTHON

```
rows = [
    {"strategy": "A", "symbol": "RELIANCE", "qty": 2},
    {"strategy": "B", "symbol": "DABUR", "qty": 3},
    {"strategy": "A", "symbol": "RELIANCE", "qty": 1},
]
totals = {}
for row in rows:
    key = (row["strategy"], row["symbol"])
    totals[key] = totals.get(key, 0) + row["qty"]
assert totals[("A", "RELIANCE")] == 3
assert ("A", "DABUR") not in totals
```

The tuple defines ownership. Replacing it with just `symbol` would merge allocations from different strategies. This example groups already-validated rows; it is not a database transaction or a broker position reconciliation algorithm.

A comprehension is convenient when it remains readable. Use an ordinary loop when you need several validation steps or detailed errors. Clever one-liners are not an engineering objective. Learn dictionary iteration through `.items()`, set intersection for membership checks, and a deque for processing oldest events.

Exercise: Add account and product to the key. Create two clients with the same strategy name and symbol. Prove they remain separate. Explain why a string such as `account:strategy:symbol` needs careful escaping if any field can contain the separator.

Chapter 4 Functions contracts and precise money

A function separates an operation from the details of its caller. Parameters are inputs; a return value is an output. A pure function has no externally visible side effects and depends only on its inputs. Pure sizing and stop calculations are easier to test than functions that also read Redis, call Dhan, and send Telegram messages.

Floating point is suitable for many measurements, but prices and tick-size alignment benefit from decimal arithmetic. Construct `Decimal` from text, not from a binary float whose rounding error has already occurred.

PYTHON

```
from decimal import Decimal, ROUND_CEILING, ROUND_FLOOR

def align_limit(price: Decimal, tick: Decimal, side: str) -> Decimal:
    if not price.is_finite() or not tick.is_finite():
        raise ValueError("Price and tick must be finite")
    if price <= 0 or tick <= 0 or side not in {"BUY", "SELL"}:
        raise ValueError("Invalid order-price inputs")
    rounding = ROUND_CEILING if side == "BUY" else ROUND_FLOOR
    units = (price / tick).to_integral_value(rounding=rounding)
    result = units * tick
    if result <= 0:
        raise ValueError("Aligned price must be positive")
    return result

assert align_limit(Decimal("100.03"), Decimal("0.05"), "BUY") == Decimal("100.05")
```

This rounds an aggressive buy limit upward. It is not a universal rounding rule for every order type. Circuit limits and an absolute slippage cap must still be checked after rounding. If rounding upward violates the cap, reject the request rather than silently paying more. Tick size comes from the correct instrument metadata; do not assume every stock uses 0.05.

For capital sizing, `floor(capital / price)` respects the stated allocation before fees and margin constraints. Your earlier requirement to buy at least one share even if it exceeds capital is a different policy. Name it `allow_minimum_one_over_budget`; do not conceal it inside arithmetic. With capital 10,000 and price 12,000, strict sizing returns zero and skips. The alternate policy returns one and explicitly exceeds the budget.

Avoid mutable default arguments such as `def add(x, items=[])`. Defaults are evaluated once, not per call. Use `None` and create a list inside. Use keyword-only parameters for safety-sensitive options, and annotations to document expectations. An annotation alone does not validate a web request.

Build: Implement strict sizing, optional minimum-one sizing, target, stop, and P&L as pure functions. Test BUY and SELL, zero, missing, infinity, NaN, invalid sides, and very small ticks. Explain the policy differences before optimizing anything.

Worked lesson weighted average and invariants

PYTHON

```
from decimal import Decimal

def average_fill(fills):
    if not fills:
        raise ValueError("No fills")
    quantity = 0
    value = Decimal("0")
    for qty, price in fills:
        if type(qty) is not int or qty <= 0:
            raise ValueError("Invalid fill quantity")
        if not price.is_finite() or price <= 0:
            raise ValueError("Invalid fill price")
        quantity += qty
        value += qty * price
    return quantity, value / quantity
```

```
qty, average = average_fill([
    (2, Decimal("100")), (3, Decimal("102"))
])
assert qty == 5 and average == Decimal("101.2")
```

Trace the running quantity and value after each iteration. The result is not the unweighted average 101 because the two price levels have different quantities. Use the returned confirmed quantity rather than the original requested quantity.

A useful invariant is that an average of positive-weight prices lies between the minimum and maximum prices. Write a property-based test for that claim. Then deliberately feed a duplicate execution twice and observe that arithmetic cannot identify the duplicate by itself. Deduplication belongs before aggregation.

Chapter 5 Exceptions files time and logging

Exceptions report that an operation could not meet its contract. Catch the narrow errors you understand. Catching Exception at a service boundary can prevent a process from crashing, but it must preserve an explicit failed state and alert operators. `except Exception: pass` converts failure into misleading silence.

Use `try/finally` or a context manager to release files, sessions, and locks. A leaked socket or event loop owns operating-system file descriptors; increasing `LimitNOFILE` may postpone a leak rather than repair it.

PYTHON

```
import csv
from pathlib import Path

def load_symbols(path: Path):
    with path.open(encoding="utf-8", newline="") as handle:
        for row in csv.DictReader(handle):
            yield row["symbol"].strip().upper()
```

Use parsers for CSV and JSON. Splitting CSV on commas fails when quoted fields contain commas. Avoid deserializing untrusted pickle data, evaluating expressions from request bodies, or building shell commands from symbols.

Time has multiple meanings. Store event timestamps as timezone-aware UTC. Convert to Asia/Kolkata for trading-day rules and display. Use `time.monotonic()` for durations inside one process, because wall clocks can jump. Monotonic values from different machines or restarts are not comparable persistent timestamps.

PYTHON

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

received_at = datetime.now(timezone.utc)
local = received_at.astimezone(ZoneInfo("Asia/Kolkata"))
print(local.isoformat())
```

Install `tzdata` in environments without the IANA timezone database, commonly Windows virtual environments. A weekday check is not an exchange calendar: holidays and special sessions need explicit treatment.

A log should answer what happened, to which entity, in which process, and with which correlation identifier. Log `signal_id`, `intent_id`, sanitized broker error class, and duration. Never log tokens, OTPs, entire environment files, or credential-bearing URLs. Metrics describe counts and latency distributions; logs explain individual events; traces link an operation across services.

Build: Read a small CSV, reject malformed records with row numbers, and emit structured summaries. Inject a clock function into your risk rules so tests do not depend on Saturday versus Monday.

Exit check: Explain why a current heartbeat timestamp does not prove that a stock's last tick is current.

Worked lesson exception boundaries and cleanup

PYTHON

```
from contextlib import contextmanager

@contextmanager
def tracked_resource(events):
    events.append("opened")
    try:
        yield "resource"
    finally:
        events.append("closed")

events = []
```

```
try:
    with tracked_resource(events):
        raise ValueError("simulated failure")
except ValueError:
    events.append("reported")
assert events == ["opened", "closed", "reported"]
```

finally runs while control leaves the protected block. It does not make an external action reversible. Closing a socket after an order timeout does not cancel an order at the broker.

For a real adapter, translate a provider-specific error into a domain exception with a sanitized reason, preserving the original cause through `raise DomainError(...) from exc`. At the route boundary, produce a safe response and correlation ID. At the worker boundary, persist the failed or unknown state. Do not log the full request just to make diagnosis easy.

Exercise: Raise during setup, normal work, and cleanup separately. Explain which exceptions escape. Add a test that resources are released when a coroutine is cancelled.

Chapter 6 Objects dataclasses and interfaces

An object groups state with behavior. Use it when that grouping makes an invariant easier to preserve. Do not turn every small arithmetic function into a class. Prefer composition: an execution service has a broker and a repository; it is not a subclass of every broker and database.

PYTHON

```
from dataclasses import dataclass
from decimal import Decimal
from typing import Protocol

@dataclass(frozen=True)
class OrderIntent:
    intent_id: str
    symbol: str
    side: str
    quantity: int
    limit_price: Decimal

@dataclass(frozen=True)
class AcceptedOrder:
    broker_order_id: str

class Broker(Protocol):
    def submit(self, intent: OrderIntent) -> AcceptedOrder: ...
```

The protocol expresses what a caller needs, not how a broker fulfills it. Dhan, Kite, and a paper broker can satisfy the same interface with different implementations. Crucially, `AcceptedOrder` contains no invented filled quantity. You need a separate fill or order-status contract.

`frozen=True` prevents assigning new attribute values, but does not recursively freeze an inner list. Use tuples where deep immutability matters. Dataclass equality compares fields; object identity asks whether two names refer to the same instance. Learn `==` versus `is`; compare values with equality and use `is None` for absence.

Class attributes are shared across instances. Accidentally defining `positions = {}` on a class can mix user state. Instance attributes belong to one object, but concurrent tasks can still access them. Encapsulation is useful only when callers cannot bypass the rules by mutating the underlying dictionary.

An abstract base class can enforce method implementation at instantiation. A protocol supports structural typing. Neither proves remote broker behavior. Contract tests bridge your Python interface and real wire-level examples.

Build: Write a paper broker that returns acceptance first and emits a fill later. Inject it into a service through a protocol. Test two instances to prove they do not share accounts accidentally.

Worked lesson dependency injection with a recording fake

PYTHON

```
from dataclasses import dataclass, field

@dataclass
class RecordingBroker:
    submitted: list = field(default_factory=list)

    def submit(self, intent):
        self.submitted.append(intent)
        return {"order_id": f"paper-{len(self.submitted)}"}

first = RecordingBroker()
second = RecordingBroker()
first.submit({"symbol": "DEMO", "quantity": 1})
assert len(first.submitted) == 1
assert second.submitted == []
```

`default_factory` creates an independent list for each object. A recording fake lets a test inspect what the service asked the broker to do. It should not pretend every request is filled; add a separate method to supply execution evidence.

Design the service constructor around small dependencies: repository, broker, clock, and notifier. That makes failure injection straightforward. If a test must patch ten global variables to construct a service, consider whether the production ownership boundaries are too broad.

Exercise: Implement a broker that raises before accepting, and another that accepts but raises before returning. Your service must distinguish the certainty available in these scenarios rather than treating both as ordinary rejection.

Chapter 7 Iterators decorators typing and Python internals

An iterable can produce an iterator; an iterator yields its next item and eventually raises `StopIteration`. A generator function containing `yield` builds such an iterator lazily. This is useful for large historical data files, but it is not free streaming if you immediately wrap the result in `list()`.

Decorators wrap behavior. Use `functools.wraps` to preserve metadata. A retry decorator around arbitrary functions is dangerous: retrying a read can be reasonable, while retrying an ambiguous order submission can place another order. A decorator must respect the side-effect semantics of the function it wraps.

PYTHON

```
from functools import wraps
from time import perf_counter

def timed(function):
    @wraps(function)
    def wrapped(*args, **kwargs):
        start = perf_counter()
        try:
            return function(*args, **kwargs)
        finally:
            print(function.__name__, perf_counter() - start)
    return wrapped
```

This is a synchronous teaching decorator. Wrapping an `async def` with it would measure coroutine creation, not awaited work. Write a separate `async` wrapper and test it with an actual awaited delay.

Learn `TypedDict` for dictionary shape, `Literal` for finite choices, `Enum` for named states, `Protocol` for behavior, and generics for reusable containers. Use `mypy` or `pyright` as a feedback tool. Runtime validation is still necessary where JSON enters the process.

Understand closures and late binding: a `lambda` in a loop can refer to the loop variable's final value.

Understand descriptors by studying how `property` controls access. Understand MRO sufficiently to read a traceback involving inheritance. Metaclasses are an advanced extension mechanism, not an expected ingredient in a trading app.

Python implementations manage memory differently. In ordinary CPython, reference counting and cyclic garbage collection both matter. A retained dictionary of tasks can keep objects alive even when you expected them to disappear. `tracemalloc` studies Python allocations; operating-system RSS also includes native allocations and mapped pages. A high RSS does not automatically mean a Python reference leak.

Use `inspect.signature` on an installed SDK method when checking compatibility. Package version, Python version, installed file location, and actual method signature are more persuasive than an example copied from a different release. Do not catch every `TypeError` and resubmit an order with fewer arguments: a bug could have happened after a side effect.

Build: Stream a million synthetic ticks through a generator, calculating count and extrema without storing them. Measure peak memory. Add a typed protocol and run your type checker. Then implement and test the `async` timing wrapper.

Worked lesson asynchronous decorators and cancellation

PYTHON

```
import asyncio
from functools import wraps
from time import perf_counter

def measured_async(function):
```

```

@wraps(function)
async def wrapper(*args, **kwargs):
    started = perf_counter()
    try:
        return await function(*args, **kwargs)
    finally:
        print(function.__name__, perf_counter() - started)
    return wrapper

@measured_async
async def sample():
    await asyncio.sleep(0.01)
    return 7

async def demo():
    assert await sample() == 7

if __name__ == "__main__":
    asyncio.run(demo())

```

The wrapper awaits the function, so its duration includes the work. The `finally` block also runs on cancellation. Do not catch cancellation and continue executing a trade unless you have a precisely defined ownership protocol.

Inspect `wrapper.__name__` with and without wraps. Then implement a generator that yields records from several files. Ask when a file handle closes if the consumer stops early. Practice explicit context management instead of assuming garbage collection happens immediately.

Chapter 8 Algorithms complexity and practical data structures

Big-O describes how resource use grows with input size. It is not a stopwatch. An $O(n)$ scan over 23 sectors can be simpler and faster in practice than maintaining a complex distributed ranking structure. A dictionary lookup is expected $O(1)$, but network latency dominates a Redis lookup compared with an in-process dictionary.

For a rolling window use `collections.deque`, not repeated `list.pop(0)` which shifts elements. For expiring breakout watches, a heap can expose the next expiry efficiently. Keep a dictionary of active watch versions so obsolete heap entries can be discarded safely. A heap alone does not solve cancellation or replacement.

Sorting n sectors is $O(n \log n)$. For tiny n , sort the validated snapshot and use stable tie-breaking. For top- k from a large dataset, compare a heap-based $O(n \log k)$ approach. First exclude missing and stale values; treating missing as zero invents a ranking.

Practice these algorithms using project problems: binary search over sorted candle timestamps; sliding-window counts for login abuse; hashing for event deduplication; BFS for service-dependency analysis; intervals for session windows; prefix sums for cumulative depth quantity. Learn recursion with a base case, then identify where an iterative version is clearer.

Worked example: asks are 50 shares at 100.10 and 80 shares at 100.20. A buy of 100 needs the second level if the snapshot remains available. Cumulative quantities are 50 and 130. The relevant level is 100.20, not last-traded price. This is a linear scan of a short depth array; a search tree is unnecessary.

Build: Benchmark a list versus deque for 100,000 oldest-item removals. Implement top-sector ranking with deterministic ties. Explain worst-case space as well as time. Test empty input and repeated timestamps.

Part One review

Without looking at source, explain mutable defaults, aliasing, Decimal, UTC versus elapsed time, exceptions, context managers, generators, protocols, and $O(n \log n)$. Write tests for the sizing function from memory. The official [Python tutorial](#) is a companion reference for language details; this book supplies the project exercises and learning sequence.

Worked lesson a bounded rolling window

PYTHON

```
from collections import deque

def rolling_means(values, width):
    if width <= 0:
        raise ValueError("Width must be positive")
    window = deque()
    total = 0.0
    for value in values:
        window.append(value)
        total += value
        if len(window) > width:
            total -= window.popleft()
        if len(window) == width:
            yield total / width

assert list(rolling_means([1, 2, 3, 4], 3)) == [2.0, 3.0]
```

Each item is appended and removed at most once, giving $O(n)$ processing and $O(\text{width})$ storage. This example uses floats for a statistical illustration, not final price settlement. Recomputing each window sum would be $O(n \text{ times width})$.

Exercise: Add timestamps and reject out-of-order samples under a stated policy. Then support a time window rather than a fixed count. Explain why "last 20 ticks" and "last 20 seconds" are not equivalent for instruments with different activity.

Part Two APIs Databases and Authentication

Chapter 9 HTTP and API contracts

HTTP is a request-response protocol. A request has a method, path, headers, and optionally a body. A response has a status, headers, and optionally a body. JSON is a representation of data, not a guarantee of success. The earlier browser error `Unexpected token I` happened because code tried to parse plain `Internal Server Error` as JSON.

Design an API contract before a route function. State who may call it, which inputs are accepted, which durable effect occurs, which response means success, and whether repeating the request is safe. `GET` should not place orders. A webhook `POST` that returns `202 Accepted` means work was accepted for processing, not that a trade filled.

Use `400` for malformed application requests, `401` for missing or invalid authentication, `403` for an authenticated caller lacking permission, `404` for unavailable resources, `409` for state conflicts, `422` for schema validation, `429` for limits, and `503` for temporary inability to serve. Do not force every failure into HTTP `200` just because the browser can parse it.

JSON

```
{
  "ok": false,
  "error": "CONFIG_NOT_FOUND",
  "detail": "No active configuration matches this alert",
  "request_id": "example-request-17"
}
```

Keep public errors useful but free of passwords, stack traces, SDK URLs, SQL, and internal paths. Log the detailed diagnostic under the request ID after redaction. A reverse proxy can still return HTML errors even if your application always returns JSON; the frontend must handle both.

An endpoint like `/api/positions?user_id=2` must not trust the requested user ID as authorization. Derive the authorized account from the session and validate any requested scope against it. This remains relevant on a single-user installation because public URLs are still attacker-controlled inputs.

Build: Write an OpenAPI-style table for create strategy, receive signal, list positions, request exit, and read feed health. Include one happy response and three failures for each. Set body-size, symbol-count, and string-length limits.

Exit check: Explain why an HTTP `200` from `/api/broker-status` says nothing about whether its response body reports `ticker_connected: false`.

Worked lesson documenting one endpoint completely

For `POST /signals`, write a contract before its implementation: the caller presents a webhook credential; the account is resolved server-side; the payload contains source event identity, alert name, symbols, and source time; a bounded durable job is written; the response contains a receipt ID. Repeated delivery of the same event must not produce a new entry.

BASH

```
curl -i -X POST http://127.0.0.1:8015/api/signals \
  -H 'Content-Type: application/json' \
  -d '{"event_id":"practice-01","strategy":"Practice","symbol":"DEMO","price":"100"}'
```

This command targets the unauthenticated local paper lab only. Its supplied synthetic price is not a production webhook trust model. Save `Practice` first. Repeat the request and compare receipt identity, then change price while retaining the ID and observe conflict.

A webhook sender may not supply a convenient unique ID. Define your deduplication window and canonical payload identity carefully; hashing only the symbol forever would suppress legitimate future entries. Bound replay by strategy, account, event time, and known delivery semantics rather than assuming any hash makes a request safe.

Chapter 10 FastAPI validation and dependency injection

FastAPI connects HTTP input to Python functions. Pydantic models validate structure and constraints. Dependencies provide services such as an authenticated principal, database session, or broker interface. Dependency injection makes tests easier because you can replace the broker without replacing the business rules.

PYTHON

```
from decimal import Decimal
from typing import Literal
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, Field

app = FastAPI()

class StrategyInput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    name: str = Field(min_length=1, max_length=120)
    side: Literal["BUY", "SELL"]
    quantity: int = Field(strict=True, gt=0, le=10000)
    stop_pct: Decimal = Field(gt=0, lt=100, allow_inf_nan=False)
    retry_count: int = Field(strict=True, ge=0, le=3)

@app.post("/validate-strategy")
def validate_strategy(payload: StrategyInput):
    return {"ok": True, "strategy": payload.model_dump(mode="json")}
```

This isolated validation example does not authenticate or persist anything. `extra="forbid"` catches misspelled settings rather than silently ignoring them. `strict=True` prevents quantities such as boolean `True` from being treated as one. Decide whether numeric strings are accepted at your boundary, and test that decision.

Use application lifespan to acquire and close long-lived clients. Do not open a new Redis connection or HTTP session on every tick if a pool can be reused. Do not store a database session as a global shared mutable transaction. The repository gives useful examples to review, but your rebuild should keep route handling separate from domain decisions.

An `async def` route runs on an event loop. Calling blocking SDK code directly inside it can freeze other work. Use a bounded thread executor for synchronous network operations or an `async` client where appropriate. Ordinary synchronous route functions are handled differently by FastAPI; understand the distinction before moving code around. [FastAPI concurrency guidance](#)

Create a test-only application factory, `create_app(settings, repository, broker)`, instead of mutating global production state. Production should fail startup when required secrets or storage are absent, not silently switch to a test backend.

Build: Add schema validation to your calculator API. Test unknown fields, missing names, boolean quantity, zero retry count, negative stop, and unexpected content types. Document whether configuration edits affect existing positions or only future entries.

Worked lesson replace a dependency in a test

PYTHON

```
from fastapi import Depends, FastAPI
from fastapi.testclient import TestClient

app = FastAPI()

def current_account():
    raise RuntimeError("Real authentication belongs here")

@app.get("/whoami")
```

```
def whoami(account=Depends(current_account)):  
    return {"account_id": account["id"]}  
  
def test_dependency_override():  
    app.dependency_overrides[current_account] = lambda: {"id": "test-1"}  
    try:  
        with TestClient(app) as client:  
            assert client.get("/whoami").json() == {"account_id": "test-1"}  
    finally:  
        app.dependency_overrides.clear()
```

The test override is explicit and removed afterward. Do not implement an internet-accessible header such as `X-Test-Admin: true` as a convenient substitute. Test-only shortcuts must never be enabled by production request input.

Exercise: Add a second dependency that checks ownership of a strategy. Return 403 for an authenticated account requesting someone else's strategy. Test both paths without replacing the authorization rule itself.

Chapter 11 Relational database design

A database is not just a place to store JSON. It can enforce facts that must remain true under concurrency. A primary key identifies a row. A foreign key ensures a referenced row exists. A unique constraint prevents two transactions from creating the same logical record. A transaction groups changes into an all-or-nothing unit.

Your existing application uses Redis extensively. For the rebuild, introduce PostgreSQL as the durable ledger and keep Redis for fast transient state and queues. This is a proposed learning architecture, not a claim that Redis is inherently unsuitable or that PostgreSQL eliminates all failure.

Separate concepts that change independently:

Entity	Key facts	Important constraint
Account	Admin ownership, broker connection reference	No cross-account reads
Strategy	Stable ID, display name, settings	Unique canonical name per account
Strategy version	Immutable settings used at decision time	Position references the version
Instrument	Broker, exchange, security ID, tick size	Unique composite identity
Signal	Source event, payload digest, timestamps	Scoped deduplication key
Order intent	Desired action and quantity	Stable intent ID
Broker order	Remote ID and submission state	Unique broker/account/order ID
Fill	Execution ID, quantity, price	Unique execution identity
Position allocation	Strategy-owned quantity and basis	No negative remaining quantity
Outbox event	Payload and delivery state	Written with the business transaction
Audit event	Actor, action, reason, timestamp	Append-only application policy

Do not store target and stop only by looking up today's mutable strategy row. If a user changes a stop from 1% to 3%, yesterday's carry position should not silently change its risk policy unless the product explicitly supports that operation.

SQL

```
CREATE TABLE strategy (
  id UUID PRIMARY KEY,
  account_id UUID NOT NULL,
  canonical_name TEXT NOT NULL,
  display_name TEXT NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT FALSE,
  UNIQUE (account_id, canonical_name)
);

CREATE TABLE fill (
  id UUID PRIMARY KEY,
  account_id UUID NOT NULL,
  broker_order_id TEXT NOT NULL,
  execution_id TEXT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  price NUMERIC(20, 8) NOT NULL CHECK (price > 0),
  executed_at TIMESTAMPTZ NOT NULL,
  UNIQUE (account_id, broker_order_id, execution_id)
```

```
);
```

This is a partial schema exercise; add account and order tables with foreign keys before production use. NUMERIC provides decimal storage. Store timestamps as TIMESTAMPTZ, but remember a trading date is a separate business attribute.

Normalize repeated facts instead of copying credentials into every strategy. JSONB can hold versioned strategy-specific options, but validate their schema and index deliberately. Do not use JSON to avoid deciding what an order means.

Build: Draw the entity relationships. Create migrations with Alembic and SQLAlchemy after first writing equivalent SQL manually. Add unique constraints, check constraints, and indexes. Make the database reject duplicate fills even when two workers race.

Worked lesson querying and indexing the ledger

SQL

```
CREATE INDEX fill_account_time
ON fill (account_id, executed_at DESC);

SELECT broker_order_id,
       SUM(quantity) AS filled_quantity,
       SUM(quantity * price) / SUM(quantity) AS average_price
FROM fill
WHERE account_id = $1
GROUP BY broker_order_id;
```

\$1 represents a bound parameter in this SQL illustration; use the placeholder convention of your chosen driver. The query aggregates fills for one account. An index on (account_id, executed_at) supports account/time access patterns, but does not automatically optimize every grouping query. Read EXPLAIN on realistic data before adding more indexes.

Every index consumes storage and adds work to writes. A wide JSON document with an index on every possible property is not a substitute for deliberate access patterns. Use foreign keys for ownership relationships and a unique execution identity to prevent double accounting.

Exercise: Populate 100,000 synthetic fills, inspect query plans before and after an index, and measure execution time in a warmed and cold-ish cache scenario. Record that timings are machine- and data-dependent.

Chapter 12 Transactions concurrency and the outbox

Consider two requests that both read "no position" and then submit an entry. An application-level check alone does not prevent both. You need a serialized decision, durable reservation, or other concurrency control at the ownership boundary. A transaction cannot roll back an order already submitted to an external broker.

PostgreSQL isolation determines which changes a transaction can observe. Row locks can serialize updates to an existing row. They do not lock a nonexistent row in the same simple way, so unique constraints or explicit account/strategy guard rows matter. Serializable transactions can fail and require retry of the database transaction; never automatically replay external broker side effects inside that retry. [PostgreSQL transaction isolation](#)

For a manual exit, first atomically reserve the position for an exit intent. Commit that intent and an outbox row together. A worker later reads the intent and talks to the broker. Its crash recovery is based on the durable intent ID and broker evidence. This separates database atomicity from uncertain external execution.

The transactional outbox solves a specific problem: committing a state change and then crashing before publishing its event. Write both in one database transaction. A publisher sends pending outbox rows and marks them sent. It may publish twice if it crashes after sending but before marking. Consumers must deduplicate by event ID. Outbox does not magically create exactly-once broker execution.

Use optimistic concurrency for editable settings: update where `version = expected_version`, increment the version, and return conflict if another edit won. This avoids one browser tab overwriting another's changes unnoticed.

Learn parameterized SQL. `WHERE name = ?` or driver-specific placeholders keep data separate from SQL structure. Do not use f-strings to inject names into a query. SQLAlchemy's expression API provides parameters, but raw SQL inside it still requires care.

Build: Launch two concurrent entry requests and prove exactly one local intent is reserved. Crash a publisher after send but before acknowledgment and prove duplicate consumption does not duplicate position quantity. Add a migration rollback exercise on a disposable database, then explain why destructive rollback may not be safe for real fills.

Worked lesson a conditional transition

SQL

```
UPDATE position_allocation
SET state = 'EXIT_REQUESTED', version = version + 1
WHERE id = $1
    AND account_id = $2
    AND state = 'OPEN'
    AND version = $3
RETURNING id, version;
```

This example assumes your own position table with those columns. One returned row means this transaction won the state transition. Zero rows can mean a competing update, a closed position, or the wrong account; classify it using an authorized follow-up read. Do not call the broker before successfully reserving the intent.

Inside the same transaction, insert an outbox event describing the exit request. Commit before a separate worker performs external I/O. If the worker loses the broker response, preserve UNKNOWN and reconcile. The database transaction does not extend across the internet to the broker.

Exercise: Use two independent database connections and a barrier to issue the update concurrently. Exactly one should reserve the exit for the expected version. Demonstrate why an in-process Python lock does not protect another process.

Chapter 13 Redis as a cache queue and coordination tool

Redis stores typed data structures: strings, hashes, lists, sets, sorted sets, and streams. A key name is a convention, not access control. Scope it with environment and account. `paper:account:17:tick:NSE_EQ:1333` is clearer than a global `1tp` key. Different logical database numbers are not a strong tenant isolation boundary.

TTL is appropriate for transient quotes and session expiry, not for deleting the only copy of an open position. Expired cache data should cause a fresh fetch or an explicit unavailable state, not a fabricated zero price. A daily rollover should archive dashboard history while preserving carry positions and audit records.

Pub/Sub delivers to active subscribers; it is not a durable backlog. Lists can implement queues but need a processing list and recovery policy. Streams provide consumer groups and pending entries. Whichever mechanism you choose, model worker crashes, redelivery, poison messages, and retention explicitly. Pin and consult your Redis version's command documentation before implementing recovery features.

A lock with `SET key token NX PX duration` has an owner token and expiry. Release it only if its stored token still equals yours, using an atomic script. Otherwise an old worker can delete a new worker's lock. Lease renewal can fail. For database writes, use a fencing version when possible. Brokers generally cannot validate your fencing token, so unknown submissions still require reconciliation and conservative ownership rules.

Connection pools avoid repeated handshakes. Pipelining reduces round trips, but is not automatically a transaction. Measure network latency, command latency, queue wait, and event-loop lag separately before blaming Redis. Your previous `AuthenticationError` was a credentials mismatch, not proof of a slow datastore.

Build: Implement a quote cache with a receipt timestamp and TTL; a durable signal queue with a dead-letter path; and a session store. Test restart, lost connections, password mismatch, duplicate delivery, and a lease expiring during a slow operation. Keep critical ledger data outside any eviction policy that can discard it unnoticed.

Worked lesson safely releasing a Redis lease

LUA

```
-- KEYS[1] is the lease key; ARGV[1] is this worker's token.
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0
```

Compare-and-delete must be atomic. A separate GET and DEL can race: your old lease expires, another worker acquires it, then your DEL removes its lease. The script avoids that release bug but does not prevent a paused old worker from making an external call after expiry. That is why lease ownership and uncertain order reconciliation remain separate design problems.

Exercise: Use a disposable Redis instance. Acquire a short lease with token A, let it expire, acquire with token B, and run the release script with A. Assert B remains. Then disconnect renewal and observe how the worker stops accepting new work while resolving any in-flight uncertainty.

Chapter 14 Email ownership and secure login

An email address can serve as an identifier. It is not a password and does not prove ownership until verified. An administrator allowlist answers "which identity is permitted?" Email verification answers "can this person receive mail at this address?" Password and TOTP answer different authentication questions.

For your single-admin product, bootstrap must not be first-public-visitor-wins. Provision the allowed email and a one-time bootstrap secret out of band, or keep setup restricted to a trusted local/IAP session. Create the admin atomically. Once configured, setup must remain closed until an authorized backend recovery operation resets it.

Use a maintained password-hashing library and a contemporary password KDF such as Argon2id; choose work parameters by measuring your server and checking current guidance. Password hashes are not reversible encryption. Broker credentials and TOTP seeds must be recoverable by the application and therefore require protected encryption instead. Rate-limit password work so expensive verification cannot exhaust the server.

[OWASP authentication guidance](#)

Email verification flow

1. Validate and canonicalize the address using a deliberate policy. Do not invent Gmail dot-removal rules for every domain.
2. Return a generic response so public callers cannot enumerate registered accounts.
3. Generate a cryptographically random, high-entropy token for a verification link.
4. Store a digest, purpose, account binding, creation time, expiry, and consumed time.
5. Send an HTTPS link through an email provider, using a trusted configured base URL rather than an untrusted Host header.
6. On confirmation, atomically mark the token consumed and the account verified.
7. Reject reuse, expiry, purpose mismatch, and a different account binding.

PYTHON

```
import hashlib
import secrets

token = secrets.token_urlsafe(32)
token_digest = hashlib.sha256(token.encode("utf-8")).hexdigest()
assert len(token_digest) == 64
```

Only the digest belongs in the token table. This unkeyed digest is reasonable for a high-entropy random token; a six-digit OTP is a different case because its entire space is small. For OTPs, use a server-secret HMAC bound to challenge ID, purpose, and account, plus atomic attempt limits and expiry. Never log either token type.

Email scanners may prefetch links. Prefer a landing GET that does not consume the token and an explicit confirmation POST. Avoid third-party resources on the landing page, use an appropriate Referrer-Policy, and avoid retaining tokens in analytics or proxy query logs. Password-reset tokens must not be interchangeable with email-verification tokens. [OWASP reset-token guidance](#)

Sending mail reliably

Implement an email-provider interface. Use a local fake inbox in tests and a sandbox account in staging. SMTP acceptance means the receiving server accepted the message, not that it reached the inbox. Retry transient sending failures with a bounded notification queue; do not block order processing while email is down.

Configure SPF for permitted senders, DKIM signing, and a staged DMARC policy for your domain. Monitor delivery failures and complaints. These controls improve domain authentication, not recipient identity. Compute Engine restricts some outbound SMTP traffic; use a documented provider integration or supported authenticated submission path rather than assuming port 25 works. [Google Cloud mail guidance](#)

Build: Implement verification and password reset against a fake email sender. Test resend limits, email enumeration, token reuse, concurrent redemption, wrong purpose, provider failure, and link scanning. A failed email send must never expose the OTP as a debugging fallback.

Worked lesson binding an email OTP to its purpose

PYTHON

```
import hashlib
import hmac
import secrets

def otp_digest(key, challenge_id, purpose, account_id, code):
    message = "\x00".join([challenge_id, purpose, account_id, code])
    return hmac.new(key, message.encode(), hashlib.sha256).hexdigest()

key = secrets.token_bytes(32)
code = f"{secrets.randbelow(1_000_000):06d}"
stored = otp_digest(key, "challenge-7", "verify_email", "account-1", code)
candidate = otp_digest(key, "challenge-7", "verify_email", "account-1", code)
assert hmac.compare_digest(stored, candidate)
assert stored != otp_digest(key, "challenge-7", "reset_password", "account-1", code)
```

This is a digest demonstration, not a complete authentication system. Keep the HMAC key outside the database, validate that identifiers cannot contain the delimiter or use structured serialization, and enforce expiry, rate limits, and atomic single use. A six-digit secret is easy to exhaust without online attempt limits.

Exercise: Write a transaction that increments a failed-attempt count without extending original expiry. Two correct simultaneous submissions must not both create sessions. Use a fake mail sender that records messages in test memory; never expose codes through a production response.

Chapter 15 TOTP sessions and broker authorization

TOTP calculates a short code from a shared seed and time. Use a maintained library such as PyOTP; do not implement cryptography yourself. Display the enrollment QR only after password verification. Keep enrollment pending until a valid code is confirmed. Encrypt the seed, never log it, and restrict access to it.

On login, verify the password first, then challenge for TOTP, then issue a full session. A pending MFA session must not access trading routes. Record the accepted time step atomically to prevent simultaneous reuse where your policy requires replay protection. Keep the tolerated clock window narrow and monitor time synchronization.

Recovery is part of authentication, not a bypass. Generate one-use recovery codes, show them once, store their digests, rate-limit attempts, and audit use. A developer reset must require trusted VM/IAM access, revoke existing sessions, remove pending challenges, and document who approved it. Do not delete strategy or position records while resetting authentication.

Use opaque, unpredictable session tokens stored server-side by digest. Cookies should have `HttpOnly`, `Secure` under HTTPS, a constrained path, and an appropriate `SameSite` policy. Rotate the session after authentication and privilege changes. Revoke it on logout. Check authorization for both HTTP and WebSocket routes.

Cookie-authenticated state changes need CSRF defenses. CORS controls browser cross-origin reading; it is not authentication and does not stop non-browser attackers. Validate WebSocket origins and authenticate the handshake. A terminal `curl` does not inherit the browser's session, explaining `ADMIN_AUTH_REQUIRED` even after browser login.

Keep application login separate from broker authorization. A logged-in administrator is allowed to manage the app but may still have an expired Dhan token. Broker consent callbacks must bind to a short-lived server-side pending flow and the intended account. Do not consume any callback token on behalf of whichever user ID appears in a query. Only use state or PKCE mechanisms the provider actually supports; add a secure local transaction binding when designing around provider constraints. Check the current [Dhan authentication contract](#) before implementing its consent flow.

Build: Draw the states `UNCONFIGURED`, `PASSWORD_SET`, `MFA_PENDING`, `ACTIVE`, and `RECOVERY_REQUIRED`. Test every forbidden transition. Verify that neither email verification nor a successful broker login accidentally grants an admin application session.

Worked lesson designing recovery without erasing trades

Separate `admin_identity`, `admin_sessions`, `mfa_enrollment`, and trading state in storage. A recovery operation should revoke the identity's sessions and MFA seeds under an audited authorization path, but leave fills, positions, strategy history, and broker reconciliation evidence intact.

TEXT

```
Trusted operator verifies reset authorization
-> disable new admin sessions
-> revoke existing sessions and pending MFA challenges
-> mark identity recovery required
-> record audit event without secrets
-> issue a bounded out-of-band setup opportunity
```

Require a fresh password check for changing a TOTP seed. Protect the enrollment QR from caching and third-party analytics. Reject the same time-step code twice if replay protection is part of the policy, using an atomic compare/update, not two independent reads.

Exercise: Sign in in two browsers, perform a recovery operation in the test environment, and show both sessions lose access. Confirm an open paper position still exists and its risk worker still processes ticks. Security recovery should not accidentally stop position accounting.

Part Three Trading State and Distributed Systems

Chapter 16 Model the lifecycle before writing the broker call

The safest place to begin is a state machine. An order is not a boolean success flag. A local intent can be CREATED, SUBMITTING, ACCEPTED, PARTIALLY_FILLED, FILLED, CANCEL_PENDING, CANCELLED, REJECTED, or UNKNOWN. Preserve the broker's raw status alongside your normalized status because different brokers use different vocabulary.

UNKNOWN is essential. If a submission times out after the broker receives it, the application cannot infer rejection. Immediately repeating the submission may create another order. Persist the intent before submission; persist the broker order ID when available; reconcile using supported broker lookups and correlation identifiers. An identifier field is not necessarily a provider-enforced idempotency key.

The position changes when fills are confirmed, not when the order endpoint returns an ID. Suppose quantity 10 is accepted, four shares fill, and the rest remain open. Your position is four shares. If you cancel the remainder, wait for terminal evidence and recheck fills before creating a new order. A cancellation request can race a fill.

TEXT

```
signal -> validated decision -> durable intent -> broker acceptance
                                         -> confirmed fills
                                         -> position ledger
timeout -----> unknown -> reconcile
```

Every fill needs deduplication. A trade can appear in a live update, a trade-book poll, and a restart reconciliation. Those are three observations of one fact, not three new purchases. Prefer broker execution identifiers where available. Do not infer a new fill from a repeated cumulative quantity without calculating and validating the delta.

Separate requested, cumulative filled, cancelled, and remaining quantities. Never reset cumulative filled to zero when a later retry fails. Persist every child order under the same intent. Quantity invariants should be executable tests, for example $0 \leq \text{total_confirmed_fill} \leq \text{requested_quantity}$ for a simple nonsliced intent.

Build: Write a paper broker scenario that accepts an order, fills four shares, loses the update, and later reports the fill in the trade book. Deliver the report twice. Your position must still contain four shares. Explain what evidence is needed before submitting the remaining six.

Worked lesson cumulative fills across retries

Suppose intent I requests ten shares. Child order A fills four and is confirmed cancelled for its remaining six. Child order B requests six, fills two, and is then rejected for the remainder. The final confirmed position is six shares, not zero, two, ten, or sixteen. Requested quantities and confirmed executions belong in separate fields.

PYTHON

```
executions = [
    ("A", "fill-1", 4),
    ("B", "fill-2", 2),
    ("A", "fill-1", 4),
]
seen = set()
confirmed = 0
for order_id, execution_id, quantity in executions:
    key = (order_id, execution_id)
    if key not in seen:
        seen.add(key)
        confirmed += quantity
assert confirmed == 6
```

The in-memory set is only a teaching device. Use a durable unique constraint in the ledger. A process restart must not forget which fills have already been applied.

Exercise: Add a fill arriving after a cancellation request but before cancellation confirmation. Explain why submitting a replacement at the moment you send cancel can overfill the intent. Write an explicit resolution policy before coding a retry loop.

Chapter 17 Classic strategy math and exit behavior

Classic mode uses configured direction and percentage rules. A candidate signal does not imply strategy-candle confirmation unless that extra filter is explicitly enabled. Treat each filter as a separately testable decision with a reason code.

For a BUY at entry E , target is $E * (1 + \text{target_pct} / 100)$ and initial stop is $E * (1 - \text{stop_pct} / 100)$. For SELL, reverse the signs. With entry 100, target 2%, and stop 1%, BUY exits at or above 102 or at or below 99. SELL exits at or below 98 or at or above 101. These are trigger conditions, not promises of execution at those prices.

The entry should be the weighted average of confirmed fills. If two shares fill at 100 and three at 102, average entry is $(2*100 + 3*102)/5 = 101.20$. Broker fees, taxes, and realized adjustments require separate fields; gross marked-to-market P&L is not net account profit.

Trailing and cost stops

A BUY trailing stop uses the highest observed valid price since entry; a SELL uses the lowest. For BUY, the effective stop must not decrease when price falls. For SELL it must not increase. A fresh tick receipt timestamp does not guarantee a valid market timestamp, so validate source and ordering before updating extrema.

PYTHON

```
from decimal import Decimal

def long_effective_stop(old_stop, high_water, trail_pct):
    candidate = high_water * (Decimal("1") - trail_pct / Decimal("100"))
    return max(old_stop, candidate)

assert long_effective_stop(Decimal("101"), Decimal("103"), Decimal("2")) == Decimal("101")
```

Apply this function only when trailing is enabled. If cost SL is enabled with initial SL 0.6% and RR 2, a 1.2% favorable move can move the stop to entry. Do not derive the trigger from an already-trailed stop, because that changes the original risk unit. A price stop at entry is not financially break-even after costs.

Pyramiding adds confirmed quantity, not merely requested quantity. Decide whether targets rebase to average entry and whether a new stop can ever loosen. Preserve a monotonic protective stop unless the user has explicitly authorized a different policy. Track number of completed adds separately from pending add intents.

Multiple exit rules can fire on one tick. Evaluate reasons deterministically and reserve one exit intent per owned position. A target rule, alert exit, manual exit, and trailing stop must not each send a full sell. Partial exit fills reduce remaining quantity; they do not mark the entire position closed.

Build: Test BUY and SELL target equality, stop equality, trailing disabled, cost-only mode, a gap beyond stop, two simultaneous exit requests, partial exits, and pyramid fills at different prices. Add the regression: a BUY stop that reached 101 must never return to 100 after a lower tick.

Worked lesson a target and trailing replay

BUY entry is 100, target is 110, fixed stop is 98, trailing distance is 2%. Ticks are 100, 103, 105, 104, 102. At 105 the trailing line reaches 102.90. At 104 it remains 102.90. At 102 the exit trigger fires. The simulator can fill at 102; it must not claim an execution at 102.90 merely because that was the trigger.

PYTHON

```
prices = [100, 103, 105, 104, 102]
high = 100
stop = 98
history = []
for price in prices:
    high = max(high, price)
    stop = max(stop, high * 0.98)
    history.append((price, stop, price <= stop))
assert history[-1][2] is True
assert history[3][1] == history[2][1]
```

This float-based trace illustrates the sequence; use the Decimal domain implementation for price-sensitive calculations. Repeat with a SELL and verify the stop moves downward only. Then disable trailing and verify the fixed stop remains unchanged even though the high-water mark can still be recorded for later analysis.

Chapter 18 Configuration ownership and the full feature map

Use a stable strategy ID as the owner. Alert names are external lookup keys and display strings. Define normalization once; preserve the original display name; reject genuine collisions rather than selecting an arbitrary configuration. An edit to an existing strategy should identify that strategy, so changing unrelated fields is not mistaken for a duplicate creation.

For alert-based exit, match account, configured exit-alert identity, owning strategy, instrument, and open quantity. Strategy A's exit alert containing DABUR must not close Strategy B's DABUR position. If broker netting aggregates the same instrument across strategies, keep a local allocation ledger and reconcile aggregate broker quantity against the sum of those allocations. Account-level exits are a separate, explicitly broader operation.

Feature	Rebuild exercise	Required failure test
Enabled strategy	Immutable configuration versions	Disabled alert cannot enter
Direction and product	Validate supported combinations	Unsupported CNC short rejected
Capital or fixed size	Separate sizing policies	Zero, missing and over-budget behavior
Entry window	Exchange timezone and date	Exact start/end boundaries
Daily trade limit	Atomic reservation and release policy	Simultaneous alerts at the limit
Sector filter	Direction-aware valid snapshot	SHORT chooses lowest changes
Breakout	Anchored candle and persistent watch	Expiry and duplicate wakeups
Target and SL	Pure trigger calculations	Gap and partial exit
TSL and cost SL	Independent toggles	Stop cannot loosen
Pyramiding	Child intents and average basis	Duplicate add request
CNC carry	Durable ownership and restart restore	Missing holdings is not assumed sale
Account P&L exit	Explicit product/account scope	Misleading UI scope rejected
Kill switch	Block entries without blocking recovery	Existing risk monitoring continues
Telegram	Async bounded notification queue	Sending failure cannot block exits
Backtest	Isolated research worker	No access to live broker adapter
Daily reset	New dashboard view and counters	Carry positions survive

For CNC carry, broker positions and holdings can represent different settlement phases. Preserve unresolved ownership until there is evidence. Do not adopt every manually purchased holding as an app-managed position. Confirm available sell quantity and broker authorization requirements before an exit. A process running overnight cannot guarantee a fill while the exchange is closed.

Daily P&L controls must define realized versus unrealized, gross versus net, account-wide versus app-only, and MIS versus CNC scope. The visible label must match the implementation. An application kill switch is not automatically the broker's account-wide kill switch; broker controls may have different prerequisites and effects.

For Telegram, store bot secrets securely, scope destinations, and send only opted-in strategy messages. Deduplicate the 09:05 IST welcome by destination and trading day. Five-minute P&L jobs should not create duplicate account messages per strategy. Label stale or unavailable MTM instead of publishing zero. These are notification semantics, not part of order acknowledgment.

Build: Implement one feature per pull request with a saved-configuration roundtrip test and a behavior test. Keep a feature matrix with implemented, tested, documented, and deferred columns. Do not mark a feature complete because its toggle appears on screen.

Worked lesson stable IDs versus editable names

JSON

```
{
  "strategy_id": "strategy-17",
  "version": 4,
  "display_name": "Morning Momentum",
  "entry_alert_name": "morning momentum scanner",
  "exit_alert_enabled": true,
  "exit_alert_name": "momentum weakness scanner"
}
```

A position references strategy-17 and the version used at entry. Renaming the display label does not change ownership. Editing the external alert name changes future matching under a documented policy. An exit event must match the configured exit relationship, not whichever alert name currently appears beside a row.

Exercise: Create A and B, each holding DEMO. Send A's exit alert. Assert only A's local allocation is reserved for exit. Then reconcile the aggregate broker quantity and prove the remaining allocation still belongs to B. Include a test where the same display name exists on a different client account.

Chapter 19 Candles breakout watches and sector ranking

A candle summarizes an interval. For a one-minute signal received at 10:30:03 IST, the previous fully closed interval is 10:29:00 through 10:30:00, usually treated as start-inclusive and end-exclusive. For five minutes at the same instant, anchor to 10:25 through 10:30. Know whether the provider labels a candle by opening time or closing time.

Fetch the required historical candle through a broker adapter, validate instrument and interval, convert timestamps correctly, and exclude the still-forming candle. Do not assume every API's interval names or timezone semantics are identical. If resampling from one minute, anchor to the exchange session and detect missing bars rather than pretending gaps contain real data.

Store the selected reference high or low when the signal creates a breakout watch. Re-fetching "previous candle" every minute changes the strategy. If the user means a rolling reference, expose that as a different mode. Define the buffer's unit explicitly: 0.10 rupees and 0.10% are very different.

For LONG, the threshold is the reference high plus the chosen buffer; SHORT uses the reference low minus it. Define whether touching the threshold qualifies or a strict crossing is required. This book uses $\geq$ for LONG and $\leq$ for SHORT as a documented exercise policy.

A watch expires at the earlier of signal-receipt time plus TTL and the strategy entry end time. With a two-minute TTL at 10:30:03, expiry is 10:32:03 unless the entry window closes earlier. Persist it so a restart neither forgets it nor extends its life. On trigger, recheck kill switch, duplicate ownership, available budget, current config policy, and market-data freshness. Use a single-consumer claim to prevent two ticks submitting two entries.

Sector percentage is $(\text{current_index_price} - \text{previous_session_close}) / \text{previous_session_close} * 100$. Previous close must be positive and identified by the correct previous trading session, not simply yesterday's calendar date. Premarket data can legitimately show LTP equal to previous close; a cached pct: 0 at 08:46 is not a live ranking at noon.

Cache the baseline; update the numerator from live validated ticks. A current-day market close must not overwrite the previous-session baseline while calculating that day's change. Separate baseline health, live-price health, and ranking completeness. For LONG sort descending; for SHORT sort ascending. Decide whether eligibility additionally requires positive or negative movement, because "lowest-ranked" does not necessarily mean negative.

Unknown sector mapping should produce an explicit policy outcome only when the filter is enabled. Instrument/security-ID resolution is a different concern and must work independently of membership in your curated sector list.

Build: Simulate a Friday-to-Monday cache, an intraday cache refresh, one missing index, equal percentages, and SHORT ranking. Add a watch at 10:30:03, advance a fake clock, and verify it cannot fire after expiry or change reference candle silently.

Worked lesson anchoring a candle and TTL

PYTHON

```

from datetime import datetime, timedelta
from zoneinfo import ZoneInfo

received = datetime(2026, 9, 18, 10, 30, 3,
                    tzinfo=ZoneInfo("Asia/Kolkata"))
current_start = received.replace(second=0, microsecond=0)
reference_start = current_start - timedelta(minutes=1)
deadline = received + timedelta(minutes=2)
assert reference_start.strftime("%H:%M:%S") == "10:29:00"
assert deadline.strftime("%H:%M:%S") == "10:32:03"

```

For general intervals, anchor to the exchange session rather than assuming every interval begins at midnight. Obtain the reference high from the completed bar, persist it with its interval, and keep the threshold constant until this watch expires. Do not treat missing candles as high zero.

Exercise: Construct sector data with changes 1.4, -2.1, 0.3, and missing. LONG top two should rank 1.4 then 0.3; SHORT bottom two should rank -2.1 then 0.3 unless your policy additionally requires a negative sign. Write that sign policy separately from ranking.

Chapter 20 Async concurrency and bounded work

Concurrency means handling overlapping work. Parallelism means executing work simultaneously. An event loop switches between coroutines at await points. It does not make blocking CPU work nonblocking. Threads can help with synchronous I/O; process workers are useful for CPU-heavy backtests and isolation. Do not rely on a particular GIL configuration as your correctness mechanism.

PYTHON

```
import asyncio

async def worker(queue):
    while True:
        job = await queue.get()
        try:
            if job is None:
                return
            await asyncio.sleep(0)
        finally:
            queue.task_done()

async def main():
    queue = asyncio.Queue(maxsize=32)
    async with asyncio.TaskGroup() as group:
        for _ in range(2):
            group.create_task(worker(queue))
        for number in range(100):
            await queue.put(number)
        await queue.join()
        for _ in range(2):
            await queue.put(None)

if __name__ == "__main__":
    asyncio.run(main())
```

The bounded queue applies backpressure. It does not persist jobs, and a worker crash in this simple example ends the task group. Learn that behavior before replacing the queue with Redis. Never call `asyncio.run` inside an already running event loop or create a new loop for each market tick.

Handle cancellation deliberately. Cancelling an await around `asyncio.to_thread` does not necessarily stop the thread or undo its network call. If the operation may have submitted an order, cancellation creates an unknown outcome requiring reconciliation. Preserve ownership until that uncertainty is resolved.

Use a semaphore to bound concurrent candle requests and a separate rate limiter for broker request quotas. They solve different problems: concurrency limits in-flight work; rate limits requests per time. Reserve capacity for exits and reconciliation so a burst of entry signals cannot starve safety work.

A queue of obsolete ticks is harmful even if nothing was dropped. Preserve original receipt timestamps so delayed work cannot appear fresh. Dashboard updates can often coalesce to the newest price. Risk checks may need ordered ticks and extrema to avoid missing a brief stop crossing. Define overflow behavior explicitly; silently dropping risk events is not acceptable.

Build: Inject a slow candle API while ticks continue. Measure tick handling delay. Cap concurrency, then compare latency before and after. Kill a task during simulated submission and verify your system records uncertainty rather than retrying blindly.

Worked lesson concurrency is not the same as a rate limit

PYTHON

```
import asyncio

async def demo():
```

```

in_flight = 0
peak = 0
semaphore = asyncio.Semaphore(2)

async def fetch(number):
    nonlocal in_flight, peak
    async with semaphore:
        in_flight += 1
        peak = max(peak, in_flight)
        try:
            await asyncio.sleep(0.01)
            return number
        finally:
            in_flight -= 1

result = await asyncio.gather(*(fetch(i) for i in range(6)))
assert result == list(range(6)) and peak == 2

if __name__ == "__main__":
    asyncio.run(demo())

```

Two concurrent calls can still create hundreds of requests per second if each finishes quickly. Implement rate limiting separately using provider-specific budgets. Reserve a portion for exits and reconciliation. In a multi-process deployment, a per-process limiter may multiply total request rate; decide where the shared account quota is enforced.

Chapter 21 Broker adapters and WebSocket evidence

A broker adapter isolates external request shapes, response validation, error normalization, and version differences. It should expose domain operations without pretending that Dhan and Kite have identical product names, instrument identifiers, depth packets, or authentication.

Read the pinned SDK source and official API contract together. Dhan's market feed uses subscription requests and binary market responses; its order-update feed is a different channel. Kite also supplies a binary market stream with separate message handling. Let the supported SDK parse protocol packets where possible, and test your normalization against sanitized fixtures. [Dhan market feed](#), [Kite WebSocket protocol](#)

Track at least these states separately: credentials valid, socket connecting, socket open, subscription requested, first tick received, latest tick fresh, reconnecting, and authentication blocked. A service heartbeat is not a broker heartbeat. A subscription request sent is not exchange acknowledgment. REST fallback must be marked as REST, not relabeled as a WebSocket tick.

Keep one connection owner per broker account where required by your architecture. Token changes must notify that owner. Recreate a stale client on credential change; do not require the operator to discover that an old token remains in memory. Maintain the active subscription set and replay it after reconnect. Deduplicate instrument subscriptions by exchange and security ID.

Use exponential backoff with jitter for transient failures. Authentication rejection should stop futile retries and request reauthentication. Do not reconnect aggressively merely because an event-based instrument has no recent trades. Outside the market session, absence of ticks is not enough to diagnose a broken socket.

Store both source and receipt time with prices. If a REST fallback is permitted, enforce its age and rate limits and make degraded status visible. Fallback may support temporary monitoring but cannot guarantee instant exits during provider or network failure.

For aggressive limits, use fresh ask depth for BUY and bid depth for SELL; inspect cumulative depth for quantity; apply a bounded buffer; align to instrument tick size; enforce circuit and slippage limits. Depth is a snapshot, not a reservation. No algorithm can guarantee instant execution. Broker rejection must retain its specific reason. [Dhan order contract](#)

Build: Maintain a capability table for each SDK version. Replay malformed packets, expired-token disconnects, duplicate subscriptions, stale ticks, and order updates with missing quantities. Assert that malformed broker data cannot create a fill. A package upgrade is a change requiring contract tests, not a routine blind install.

Worked lesson a capability matrix before an SDK upgrade

TEXT

Capability	Dhan adapter	Kite adapter
Instrument identity	exchange + ID	exchange + token/symbol mapping
Accepted order	validated order ID	validated order ID
Fill evidence	normalized trades	normalized trades
Quote freshness	source + times	source + times
Pending cancel	request + confirm	request + confirm
Token replacement	owner reload	owner reload

Fill this table from the pinned SDK and current provider API. Inspect a method signature locally:

PYTHON

```
import inspect

def inspect_method(client):
    print(type(client).__module__)
    print(inspect.signature(client.place_order))
```

This function expects a client passed by your test setup. Do not construct a live client or print its configuration solely to inspect a signature. Add a contract fixture showing the exact request your adapter emits. Test upgrades in a separate environment and compare request fields, response normalization, exception behavior, and feed callbacks before deployment.

Chapter 22 Service boundaries recovery and architecture drawings

Your target has five deployable processes: API/dashboard, alert intake, market feed, execution, and reconciliation. The sixteen logical responsibilities discussed earlier can live inside these processes as modules. Separate logical ownership first; add more processes when workload isolation, fault containment, or scaling justifies them.

TEXT

```

Browser --HTTPS--> Nginx --> API/dashboard --read--> state
Browser <--WSS---- Nginx <-- notifications <----- tick/state bus

Chartink --> authenticated intake --> durable signal queue
      |
      | strategy + decision + breakout watch
      |
      | execution owner --> broker
      | ^
      | | orders and fills
      | | reconciliation
Broker market feed --> normalized ticks --> risk
      |
      | +---- durable position ledger

Email / Telegram / analytics consume events outside the order path

```

Keep the API from directly bypassing execution ownership for manual square-off. A manual command is another durable intent. The reconciliation process observes broker facts and updates the ledger through the same quantity rules, rather than editing arbitrary Redis fields without accounting.

Think through failure windows: before enqueue, after enqueue before acknowledgment, after claim, after broker submit before recording ID, after a partial fill, and after a database commit before publishing. Write a recovery policy for each. The most dangerous interval is not necessarily the slowest one.

Delivery guarantees apply at particular boundaries. At-least-once queue delivery is compatible with deduplicated local effects. End-to-end exactly-once trading is not obtained by naming a queue "exactly once." The broker, network, ledger, and consumer all have independent failure semantics.

Start capacity planning with assumptions: candidate signals per minute, active instruments, ticks per second, concurrent browsers, broker call quota, and acceptable queue delay. Estimate average queued work with Little's Law, $L = \text{arrival_rate} * \text{time_in_system}$, when stable. Then measure p95 and p99 latency; averages hide bursts. A queue growing faster than it drains needs admission control, not unlimited RAM.

Use liveness for process viability and readiness for safe work acceptance. A disconnected broker might make new-entry readiness false while the API must remain available for status and recovery. Restarting every component on every failed dependency can amplify an outage.

Build: Draw a context diagram, container/process diagram, signal sequence diagram, and order state machine. Annotate each arrow with authentication, timeout, durability, and retry policy. Explain why dashboard WebSocket failure and Dhan WebSocket failure are independent paths.

Worked lesson capacity calculation with explicit assumptions

Assume 120 candidate jobs per minute and mean processing time 0.4 seconds. The mean arrival rate is two jobs per second. If stable, Little's Law predicts about 0.8 jobs in the system on average. This does not describe a burst of 100 simultaneous signals or a provider call that stalls for 30 seconds.

PYTHON

```
arrival_per_second = 120 / 60
mean_seconds = 0.4
average_in_system = arrival_per_second * mean_seconds
assert average_in_system == 0.8
```

Now assume 500 instruments each produce four ticks per second and each normalized event is about 250 bytes. The raw event payload rate is about 500,000 bytes per second before protocol, copies, storage, and fanout overhead. Measure actual sizes and serialization costs rather than treating this estimate as capacity proof.

Exercise: Draw the bottleneck if one worker performs three serial 0.3-second broker calls per signal. Explain what can run concurrently and what must remain ordered per account or position. Add admission limits rather than an unlimited queue.

Part Four Frontend Integration and Software Testing

Chapter 23 Browser fundamentals for a backend developer

HTML describes content and form controls. CSS controls layout. JavaScript handles events and communicates with your API. The DOM is the browser's current document tree, not the server database. A number displayed on a page may be stale even while the backend is healthy.

Use labels associated with controls, semantic buttons, keyboard focus, and readable validation messages. Disabled controls should express actual unavailable actions. A hidden field is not a security boundary. User IDs and prices submitted by the browser remain untrusted.

Use server-calculated risk values for display. The browser can preview a calculation, but the execution path must validate and calculate again. Prevent duplicate submissions for usability, then enforce idempotency on the server for correctness. Closing a browser tab must not stop server-side risk monitoring.

JAVASCRIPT

```
async function fetchJson(url, options = {}) {
  const response = await fetch(url, options);
  const text = await response.text();
  let data;
  try {
    data = text ? JSON.parse(text) : null;
  } catch {
    throw new Error(`Unexpected server response (${response.status})`);
  }
  if (!response.ok || data?.ok === false) {
    throw new Error(data?.detail || `Request failed (${response.status})`);
  }
  return data;
}
```

This handles proxy-generated non-JSON without exposing the full raw response. Production also needs timeout/cancellation, session-expiry behavior, and accessible error display. Retrying a GET differs from retrying a square-off POST.

Render untrusted symbols and error messages with `textContent`, not HTML concatenation. Avoid placing secrets into URLs, DOM attributes, analytics events, or browser local storage. A webhook copy button can use the Clipboard API under its browser security requirements and provide a clearly labeled fallback when unavailable.

For mobile, test the actual layout at narrow widths. A table may scroll horizontally inside its container; the entire page should not overflow. Use one-column forms at small sizes, predictable grouping, and touch-sized controls. Do not make a modal so tall that its Save button becomes inaccessible. Preserve a visible stale-data indicator even on mobile.

Build: Implement configuration save/reload with server validation errors shown next to the relevant field. Submit zero retries and verify zero survives reload. Display an offline banner without removing the last known position data or pretending it is fresh.

Worked lesson avoiding stale responses in the browser

Two refresh requests can finish out of order. Request A starts first but takes longer; B returns a newer position, then A overwrites it. Use an entity version from the server or a request-generation guard for a single view.

JAVASCRIPT

```
let generation = 0;
async function refreshPositions() {
  const mine = ++generation;
  const response = await fetch('/api/positions');
  if (!response.ok) throw new Error('Positions unavailable');
  const data = await response.json();
  if (mine !== generation) return;
  renderPositions(data.positions);
}
```

`renderPositions` is your own DOM function. This guard only handles this view's request ordering; it does not solve out-of-order domain events. Include a server-side version or event sequence in WebSocket updates and reject older updates per entity.

Exercise: Intercept two responses in a browser test and deliver them backward. Verify a closed position cannot become open again merely because an older response arrives late. Preserve accessible error text while keeping the last known data marked stale.

Chapter 24 Unit testing as executable design

A unit test checks a small behavior with controlled inputs. It should explain a rule, not merely execute lines. Arrange the state, act once, and assert the important outcome. Test names should describe behavior such as `test_long_stop_never_decreases` rather than `test_function_7`.

PYTHON

```
from decimal import Decimal
import pytest

@pytest.mark.parametrize("capital,price,expected", [
    ("10000", "2000", 5),
    ("10000", "12000", 0),
    ("0", "100", 0),
])
def test_strict_sizing(capital, price, expected):
    quantity = int(Decimal(capital) // Decimal(price))
    assert quantity == expected
```

The parameterized example checks a mathematical policy; your production function also needs input validation and fee policy. For risk rules, use a fake clock and explicit ticks. Do not rely on today's date or a running exchange.

Fixtures provide setup and cleanup. A test database fixture should start empty and clean itself afterward. A fake broker should record calls and allow scenarios: reject, partial fill, delayed fill, timeout after acceptance, and unknown response. A mock that always returns success cannot test reconciliation. [pytest fixtures](#)

Use property-based tests for invariants: aligned prices are multiples of tick size; a long stop is monotonic; confirmed fill deduplication is idempotent; position quantity never becomes negative. Use mutation testing to ask whether changing `>=` to `>` makes a test fail. Coverage measures executed lines, not the truth of the expected result.

Build: Write a failing test before fixing a bug. For the zero-default bug, assert `retry_count == 0` after a save/reload. For quantity safety, send the same fill twice and assert the ledger is unchanged after the second delivery.

Worked lesson tests that catch the wrong operator

PYTHON

```
import pytest

@pytest.mark.parametrize("price,expected", [(101, False), (102, True), (103, True)])
def test_target_boundary(price, expected):
    assert (price >= 102) is expected
```

Changing `>=` to `>` should fail the equality case. If a mutation tool makes that change and the suite stays green, the tests do not protect the boundary rule. Add examples around threshold, invalid data, and exact expiry time.

Avoid asserting implementation details that do not matter, such as the precise number of helper function calls after a harmless refactor. Assert business evidence: confirmed quantity, deduplicated execution, preserved ownership, and a visible closed status. Use spies only where the call itself is the externally relevant effect.

Exercise: Write a deliberately wrong P&L function that ignores SELL direction. Add the smallest test that exposes it, then expand to a parameterized BUY/SELL table with costs clearly excluded or included.

Chapter 25 API and real integration tests

Integration tests check collaborating components. Use an actual disposable Redis or PostgreSQL instance when testing transactions, expiry, scripts, locking, or consumer recovery. An in-memory fake is useful for fast tests but may not reproduce atomicity, serialization, network failures, or database constraints.

Test a full vertical slice: HTTP request, validation, transaction, queued work, simulated broker event, ledger update, and response query. Assert stored evidence as well as status codes. A route returning 200 while discarding a setting is still broken.

For the bundled lab, from its directory:

BASH

```
python -m pytest -q tests/test_domain.py tests/test_api.py
```

The lab creates a fresh temporary SQLite database per test. It never imports your live application or broker SDK. Later, repeat the same contract against PostgreSQL and Redis implementations rather than assuming the lab proves their behavior.

API cases must include unauthenticated and unauthorized access, malformed bodies, unknown configuration, duplicate event IDs, conflicting reuse of an event ID, zero values, oversized input, expired sessions, and missing dependencies. Test that an error response does not include a Redis password or token-bearing URL.

For broker contract tests, pin an SDK version, intercept its HTTP transport, and validate exact serialized requests and parsed responses. Keep fixture metadata identifying API version, instrument, and capture date. Scrub credentials. A contract fixture is not a live sandbox; it detects changes you have modeled, not every future provider behavior.

Build: Run the same signal twice and prove only one paper position exists. Reuse its ID with a different payload and require conflict. Save a config, restart the app against the same test database, and retrieve it. Inject a storage error and ensure the API does not claim a durable enqueue succeeded.

Worked lesson a three boundary integration test

PYTHON

```
def test_signal_creates_one_position(client):
    config = {"name": "Practice", "quantity": 1}
    assert client.post("/api/configs", json=config).status_code == 200
    signal = {"event_id": "integration-1", "strategy": "Practice",
             "symbol": "DEMO", "price": "100"}
    first = client.post("/api/signals", json=signal).json()
    second = client.post("/api/signals", json=signal).json()
    rows = client.get("/api/positions").json()["positions"]
    assert first["position_id"] == second["position_id"]
    assert len(rows) == 1
```

This test uses the temporary-database `client` fixture in the included paper lab. It crosses HTTP validation, persistence, and position creation. It does not test Dhan, Redis, or network partitions. Naming the tested boundary prevents false confidence.

Exercise: Repeat against a PostgreSQL repository. Add a second process making the same request and assert the durable uniqueness constraint holds. Do not replace PostgreSQL with a dictionary for the test whose purpose is to verify transaction behavior.

Chapter 26 Playwright with Python

Playwright controls real browser engines. It can verify that a person can operate the UI and that the UI interacts correctly with the backend. Use accessible role and label selectors where possible. Avoid brittle selectors based on the fifth nested div or arbitrary sleep calls.

Install its Python pytest plugin in the learning environment, then install a browser:

BASH

```
python -m pip install pytest-playwright
python -m playwright install chromium
```

The included browser test starts a paper-only local API subprocess itself, saves a strategy through the form, sends a simulated signal through the API, verifies a rendered open position, applies a target-reaching simulated tick, and verifies a closed position. This is frontend-to-backend integration, not a screenshot of static HTML.

PYTHON

```
from playwright.sync_api import expect

def verify_strategy_form(page, base_url):
    page.goto(base_url)
    page.get_by_label("Strategy name", exact=True).fill("Practice")
    page.get_by_role("button", name="Save strategy", exact=True).click()
    expect(page.get_by_role("status")).to_contain_text("Saved")
```

This illustrative function assumes the lab form's other fields retain valid defaults. Use Playwright's web-first assertions, which retry until the condition or timeout, rather than checking DOM text immediately after a network request. [Playwright Python pytest guidance](#)

For production authentication tests, use an isolated test administrator and a fake email inbox. Generate TOTP from a test seed without logging production seeds. Do not disable authentication in the only browser suite; keep a separate suite for login, CSRF, session expiry, and authorization.

Capture traces and screenshots on failure. Traces may include requests, cookies, and page content, so treat them as sensitive artifacts with short retention. Run desktop and mobile viewports; also use Firefox or WebKit for cross-engine coverage when your support policy requires it.

Build: Add browser cases for expired session, a 502 HTML response, WebSocket disconnect, zero retries, form reload, and horizontal overflow. Simulate backend messages rather than opening real broker connections. Verify both visible output and backend records.

Worked lesson browser evidence beyond a screenshot

PYTHON

```
from playwright.sync_api import expect

def check_zero_retry_roundtrip(page, base_url):
    page.goto(base_url)
    page.get_by_label("Strategy name", exact=True).fill("Zero Retry")
    page.get_by_label("Retry count", exact=True).fill("0")
    page.get_by_role("button", name="Save strategy", exact=True).click()
    expect(page.get_by_role("status")).to_have_text("Saved Zero Retry")
    response = page.request.get(base_url + "/api/configs")
    assert response.ok
    strategy = next(row for row in response.json()["strategies"]
                    if row["name"] == "Zero Retry")
    assert strategy["retry_count"] == 0
```

The visible success message and persisted numeric value are both checked. In a production-auth suite, `page.request` shares the browser context's authentication state; keep isolated contexts per test user. Traces and stored authentication files are credentials-bearing artifacts and should not be committed.

Exercise: Add a tablet viewport and keyboard-only form submission. Test a server-side validation failure despite a browser form that appears valid. Then take a screenshot as supporting evidence, not as the only assertion.

Chapter 27 Fault injection performance and security tests

Reliable software is tested under failure, not only with fast successful mocks. Inject delayed responses, malformed JSON, lost order updates, duplicate fills, Redis disconnects, clock boundaries, worker termination, and cancellation during an in-flight submission. Record the expected final state before running the experiment.

Fault	Required behavior
Submission response lost	Mark unknown and reconcile; no blind new order
Consumer dies after claim	Recover only with ownership and idempotency rules
Ticks arrive late	Preserve receipt time and reject stale entry evidence
Telegram stalls	Trading path continues; notification is bounded
Dashboard disconnected	Server monitoring continues; browser shows degraded state
Database unavailable	No false success for unpersisted critical work
Two admin setups race	Exactly one bootstrap succeeds
Broker token rotates	Feed owner reloads credentials without process-wide guesswork
CNC missing from holdings	Preserve unresolved ownership and investigate positions

Measure end-to-end timestamps: received, validated, queued, claimed, priced, submitted, acknowledged, filled, and displayed. A two-millisecond function is irrelevant if its job waited five seconds in a queue. Use percentiles and error rate under a stated workload; never claim "superfast" without a reproducible measurement.

Define a load scenario before running it: for example, 50 paper signals per minute, 100 watched instruments, 1,000 synthetic ticks per second, and three dashboards. These are exercise inputs, not measured capacity of your app. Record machine size, software versions, duration, queue maximum, event-loop lag, RSS, descriptor count, and broker calls. Test a short burst and a long soak.

Do not load-test live broker order endpoints. Use a fake transport with realistic delays and errors. Avoid overwhelming a client's production VM with local stress tools during market hours.

Security tests should try cross-account IDs, session replay after logout, TOTP replay, CSRF, HTML in symbol names, malformed content types, arbitrary redirect destinations, path traversal in exports, secret-bearing error responses, and unauthorized WebSocket origins. Scan dependencies, but distinguish a vulnerability finding from a verified exploit in your deployment.

Build: Write a failure matrix with evidence links. Stop an isolated worker during every important state transition. Explain why `systemctl active` and a clean error grep cannot prove business correctness.

Worked lesson interpret a latency distribution

PYTHON

```
from statistics import median

samples_ms = [10] * 95 + [800] * 5
ordered = sorted(samples_ms)
def nearest_rank(percent):
    from math import ceil
    return ordered[max(0, ceil(percent * len(ordered)) - 1)]

assert median(samples_ms) == 10
assert nearest_rank(0.99) == 800
```

The median looks excellent while one percent of requests can still be very slow. Percentile definitions vary for finite samples, so state the method and use an appropriate histogram in production. Include timeouts and failed requests in reliability analysis rather than measuring only successes.

Exercise: Run a synthetic load test for five minutes and a soak test for an hour. Compare memory, thread count, open descriptors, queue length, and p99 latency at the beginning and end. A test that slowly accumulates resources may pass a short demonstration and fail during an entire market session.

Chapter 28 CI and evidence based release decisions

Continuous integration should run formatting, static checks, unit tests, integration tests, contract tests, and selected browser tests on every proposed change. Separate fast feedback from slower scheduled fault and soak tests. Treat flaky tests as defects; endless retries can hide a real race.

Build one immutable artifact and promote it between environments. Pin dependencies, record the Python and SDK versions, and review changes in transitive packages. Include a vulnerability scan and secret scan, but do not make their green output your entire security argument.

A release report should say what changed, which tests ran, their environment, what was not tested, known limitations, migration steps, and rollback conditions. For example, "paper broker tested; no live exchange validation" is honest. "All bugs fixed" is not a defensible test result.

Database migrations need compatibility planning. Add new optional fields before making readers depend on them. Deploy producers and consumers that tolerate the transition. Avoid deleting old data while older workers may still need it. Rollback of code and rollback of state are different operations.

For a trading deployment, pause new entries during ownership handover, preserve exit monitoring, drain or reconcile in-flight intents, restart with a known version, and check feed and queue readiness before resuming. Do not deploy two live execution owners as a conventional blue-green test without explicit fencing and account isolation.

Build: Write a CI workflow for the learning repository and deliberately introduce a failed zero-value test, a syntax error, and an accidental secret placeholder matching your scanner rule. Verify each fails for the intended reason. Present a release checklist that another person can follow without knowing your terminal history.

Worked lesson a CI job with an honest boundary

YAML

```
name: paper-lab
on: [push, pull_request]
jobs:
  tests:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: docs/learning-book/lab
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: python -m pip install -r requirements.txt
      - run: python -m playwright install --with-deps chromium
      - run: python -m pytest -q tests
```

This is a teaching workflow, not an assertion that these action tags will always be current. Production pipelines should pin reviewed immutable action revisions and periodically update them. The included browser suite falls back to installed Playwright Chromium when a Chrome channel is unavailable.

Exercise: Add separate artifacts for failure traces, restrict retention, and ensure no broker secret is available to pull-request jobs from untrusted branches. Record the test Python/package versions. A green paper-lab job is not authorization to trade live.

Part Five Google Cloud and Production Operations

Chapter 29 Cloud fundamentals and resource hierarchy

Cloud computing gives you programmable access to compute, storage, networking, identity, and managed services. It does not remove architecture decisions. A VM is still a running operating system with patching, filesystem capacity, network access, and process supervision to manage.

An organization can contain folders and projects. A project groups resources, API enablement, quota, and IAM policies. A billing account pays for linked projects. IAM answers who can do which operation on which resource; billing linkage is not itself permission to administer every workload.

For client isolation, use a dedicated project and separate identities, secrets, databases, and deployment state. Identical local usernames on separate VMs do not inherently mix clients. Copying a populated Redis database, shared broker token, production `.env`, or globally configured webhook URL can. A VM administrator can normally read code and secrets available to that VM; obfuscation is not a strong boundary against the owner of the machine.

A region is a geographical area; a zone is a deployment location within it. Mumbai is `asia-south1`, with zones such as `asia-south1-b`. A zonal outage can stop a single VM. Choosing another zone may solve a capacity error but does not create high availability for an existing one-VM deployment.

Learn the difference between quota and capacity. Quota is a project/account limit; capacity is currently available provider infrastructure. `ZONE_RESOURCE_POOL_EXHAUSTED` does not imply your Python code is broken. An under-200-GB disk performance warning does not mean Google created a 200-GB disk; inspect the actual disk resource.

Lab: In a disposable project, list enabled APIs, assigned IAM roles, regions, zones, and billing linkage. Draw which resources are global, regional, and zonal. Explain what would survive deleting a VM versus deleting its boot disk versus deleting the project.

Worked lesson inspect scope before creating resources

BASH

```
gcloud auth list
gcloud config list
gcloud projects describe "$PROJECT_ID"
gcloud services list --enabled --project="$PROJECT_ID"
gcloud compute instances list --project="$PROJECT_ID"
```

Run these read-only commands before a lab. Confirm the account, project ID, and region variables. A project display name is not necessarily its immutable ID. Avoid assuming that the Cloud Shell prompt and every command's explicit `--project` refer to the same project.

Exercise: Draw two client projects with separate billing linkage, secrets, VM identities, and data stores. Mark the developer's access explicitly. Then remove developer access in the diagram and explain how client operations and billing continue. Discuss source-code visibility honestly: a client with VM administrative access can inspect deployed source.

Chapter 30 IAM service accounts and secrets

A service account represents a workload, not a person. Attach a minimally privileged service account to the VM instead of distributing long-lived JSON keys. Local development can use Application Default Credentials or controlled impersonation. Human administrator privileges and application runtime privileges should be separate. [Google Cloud service accounts](#)

Use predefined roles where they fit. A custom role is useful when a narrow operation, such as starting and stopping a particular VM, needs fewer permissions than a broad administrator role. Understand both the permission to act on a resource and the permission to attach or impersonate a service account.

Prefer OS Login and IAP for controlled SSH rather than exposing port 22 to the whole internet. IAP access requires both appropriate IAM and a firewall path for its documented source range. A firewall rule without IAM or IAM without a network path is insufficient.

Secret Manager stores versioned secrets and provides access auditing. Grant the runtime access only to the secrets it needs. Do not put secret values into Terraform variables that will be retained in state without a deliberate protection plan. Environment variables are a delivery mechanism, not encryption. Root and sufficiently privileged process inspectors can still read them. [Secret Manager practices](#)

Encryption at rest for disks does not replace application protection of broker tokens or transport encryption. A Fernet key stored beside ciphertext protects against some accidental exposures but not a full compromise that reads both. Plan rotation and backup access together: losing the only decrypting key can make credentials unrecoverable.

Lab: Create two service accounts, one for the API and one for an analytics export. Grant the exporter access to a test bucket but not broker secrets. Prove a forbidden read fails. Record the denied audit event. Revoke permission and verify what happens to already-issued credentials and cached secrets.

Worked lesson separate deployer and runtime permissions

The deployer may need to create a VM and attach a service account. The runtime needs only its application resources. The ability to impersonate a powerful service account can become privilege escalation even when the human's direct role looks narrow.

BASH

```
gcloud iam service-accounts create paper-runtime \
  --display-name='Paper application runtime' \
  --project="$PROJECT_ID"

gcloud iam service-accounts describe \
  "paper-runtime@$PROJECT_ID.iam.gserviceaccount.com" \
  --project="$PROJECT_ID"
```

This creates an identity but grants it no application permissions. Add only the required resource-scoped bindings in the relevant lab. Do not download a JSON key simply because it is easy. Prefer workload identity on managed infrastructure and controlled impersonation for humans.

Exercise: Produce a permission matrix for deployer, API, execution, analytics, and support reader. Explain why support staff can inspect sanitized logs without decrypting broker credentials. Test a denied permission and record its expected error.

Chapter 31 Network design and a safe VM exercise

A VPC is a software-defined network. Subnets are regional. Firewall rules control permitted traffic; routes control where packets go. A public static address gives a stable endpoint, but does not make an application secure. DNS maps a domain to addresses. TLS authenticates the endpoint and encrypts transport when validated correctly.

The following Cloud Shell commands create billable learning resources. Use a new project you own, confirm its billing and budget, and do not substitute a production client project. Cloud Shell is already authenticated; a local Git Bash installation needs Google Cloud CLI authentication first. Replace the placeholder deliberately.

BASH

```
export PROJECT_ID="REPLACE_WITH_LEARNING_PROJECT_ID"
export REGION="asia-south1"
export ZONE="asia-south1-b"
export VM_NAME="trading-school-vm"
export IP_NAME="trading-school-ip"
test "$PROJECT_ID" != "REPLACE_WITH_LEARNING_PROJECT_ID" || exit 1
gcloud config set project "$PROJECT_ID"
gcloud services enable compute.googleapis.com
gcloud compute networks create trading-school --subnet-mode=custom
gcloud compute networks subnets create trading-school-subnet --network=trading-school --range=10.42.0.0/24
  --region="$REGION"
gcloud compute addresses create "$IP_NAME" --region="$REGION"
export STATIC_IP="$(gcloud compute addresses describe "$IP_NAME" --region="$REGION" --format='value(address)')"
test -n "$STATIC_IP" || exit 1
gcloud compute firewall-rules create trading-school-web --network=trading-school --allow=tcp:80,tcp:443
  --source-ranges=0.0.0.0/0 --target-tags=trading-school-web
gcloud compute firewall-rules create trading-school-iap-ssh --network=trading-school --allow=tcp:22
  --source-ranges=35.235.240.0/20 --target-tags=trading-school-web
gcloud compute instances create "$VM_NAME" --zone="$ZONE" --machine-type=e2-medium --boot-disk-size=20GB
  --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud --subnet=trading-school-subnet
  --tags=trading-school-web --address="$STATIC_IP" --no-service-account --no-scopes
  --metadata=enable-oslogin=TRUE
gcloud compute ssh "$VM_NAME" --zone="$ZONE" --tunnel-through-iap
```

The no-service-account choice is intentional for a first OS/network exercise. Grant your human identity the required IAP tunnel and OS Login permissions through an authorized project administrator before SSH. Later attach a dedicated runtime account when the app needs Secret Manager or Cloud Storage. Do not grant Owner to solve every permission error.

The one-line commands avoid the trailing-backslash whitespace problem you encountered. After every resource creation, inspect the result and stop if it failed. Shell variables disappear when a session is replaced; an empty \$VM_NAME explains many resource parsing errors.

Do not run the unauthenticated reference lab on public port 80. Initially access it using an SSH tunnel to its loopback port. Public HTTPS deployment is a later exercise after your authentication chapters are implemented and tested.

Lab: Run `df -h /`, `free -h`, `lsblk`, and `ss -ltnp` on the VM. Explain disk space versus RAM versus swap. Swap can reduce an abrupt out-of-memory failure, but disk-backed paging can severely delay a real-time application. It is not extra fast RAM.

Worked lesson follow a packet

For a browser request to your public VM: DNS resolves the name, routing reaches the external address, firewall policy permits 443, Nginx terminates TLS, the proxy connects to loopback Uvicorn, and the application checks the session. A failure at any hop can resemble "the app is down" but requires different evidence.

BASH

```
gcloud compute addresses describe "$IP_NAME" \
  --region="$REGION" --project="$PROJECT_ID"
gcloud compute instances describe "$VM_NAME" \
  --zone="$ZONE" --project="$PROJECT_ID" \
  --format='get(networkInterfaces[0].accessConfigs[0].natIP)'
```

Compare reservation and attachment. On the VM, `ss -ltnp` distinguishes a listening process from a firewall problem. A route for outbound traffic does not automatically allow inbound requests. Cloud NAT supports outbound connections for appropriate private workloads; it is not an inbound public reverse proxy.

Exercise: Deliberately use the wrong port in a local proxy configuration, then distinguish connection refused from authentication denied. Do not edit a production firewall merely to reproduce the lab.

Chapter 32 Linux process supervision and deployment

systemd supervises long-running processes. active means the service process is running, not that broker authentication, subscriptions, data freshness, and exits work. Use application readiness in addition to OS process status.

Run under a dedicated non-root account. Keep code, writable data, and secrets in separate locations. Use /opt/trading-school for code, /var/lib/trading-school for application data, and a root-owned environment file for secrets. The reference lab remains loopback-only and paper-only.

This unit is a teaching template for an authenticated rebuild, not a command to replace your deployed client service:

INI

```
[Unit]
Description=Trading School API
After=network-online.target redis-server.service
Wants=network-online.target

[Service]
Type=simple
User=trading-school
Group=trading-school
WorkingDirectory=/opt/trading-school
EnvironmentFile=/etc/trading-school.env
ExecStart=/opt/trading-school/.venv/bin/python -m uvicorn school.main:app --host 127.0.0.1 --port 8015
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
NoNewPrivileges=true
PrivateTmp=true
UMask=0077

[Install]
WantedBy=multi-user.target
```

school.main is the module you will create in your rebuild, not a module supplied in the small lab. Repeat the supervision pattern for alert, market-feed, execution, and reconciliation entry points only after those modules exist. Dependencies such as After= order startup; they do not prove Redis authentication succeeded.

Changes to a unit require `sudo systemctl daemon-reload`. Changes to an environment file require restarting the affected process to load new values; daemon-reload alone does not inject variables into it. Verify which EnvironmentFile each unit actually reads. Avoid dumping all environment values into a support chat.

Deploy an immutable release or a reviewed commit, install dependencies in the intended virtual environment, run migrations, run offline checks, then hand over workers carefully. Preserve local changes; do not use a force reset on a client's machine as a routine update command. Record the commit and deployment time.

Lab: Deliberately point a test unit at a missing module and inspect `journalctl`. Then use a wrong Redis password and distinguish import failure from authentication failure. Repair the source configuration, restart only the needed service, and confirm readiness rather than repeatedly restarting everything.

Worked lesson restart evidence

BASH

```
sudo systemctl show ashuchart-api \
  -p MainPID -p ExecStart -p EnvironmentFiles --no-pager
sudo systemctl status ashuchart-api --no-pager -l
sudo journalctl -u ashuchart-api -b -n 80 --no-pager
```

`MainPID` identifies the running process. `ExecStart` identifies the interpreter and command. `EnvironmentFiles` identifies configuration sources. None of these alone proves the application's dependencies are healthy. Inspect a sanitized readiness response and broker state separately.

A unit edit requires `daemon-reload`; an `environment-value` edit requires process replacement to load the value. Repeated restarts without understanding the failure can conceal the original error and create execution ownership contention.

Exercise: In a disposable service, log only a nonsecret configuration version at startup. Change the environment file without restart and prove the old process retains its old configuration. Restart deliberately, then verify the new version and PID.

Chapter 33 Nginx HTTPS and browser WebSockets

Nginx can terminate TLS and proxy requests to loopback Uvicorn. The dashboard WebSocket traverses this proxy; the outbound broker WebSocket does not. This explains why fixing browser 426 Upgrade Required does not by itself repair a Dhan connection.

For an authenticated rebuild, a representative proxy configuration is:

NGINX

```
# Place map in the http context, not inside server or location.
map $http_upgrade $connection_upgrade {
    default upgrade;
    '' close;
}

server {
    listen 80;
    server_name trading.example.com;
    location / {
        proxy_pass http://127.0.0.1:8015;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 120s;
    }
}
```

For Nginx versions like the 1.24 installation in your logs, explicit HTTP/1.1 and upgrade forwarding are important. A successful WebSocket handshake uses HTTP 101. The application can still deny an unauthenticated handshake. `proxy_read_timeout` does not replace heartbeat and reconnect handling. [Nginx WebSocket documentation](#)

Before using a certificate tool, replace the example domain with a domain you control and point its DNS to the reserved IP. Confirm the active server block, not merely a file that was never enabled. Follow the current [Certbot Nginx instructions](#) for your OS and installation method. A typical invocation after installation is `sudo certbot --nginx -d your-real-domain.example`; use your real registered domain, not the literal example. Check `sudo certbot renew --dry-run` and the installed renewal timer.

Enable secure cookies when HTTPS is active. Trust forwarded headers only from your actual proxy. Do not allow arbitrary internet clients to spoof trusted client-IP or scheme headers by exposing the application port publicly.

Lab: Remove the upgrade header in a disposable deployment, observe the browser failure, restore it, and verify HTTP 101 in browser developer tools. Stop Uvicorn and observe the difference: Nginx may return 502, which is an upstream failure rather than a WebSocket-only failure.

Worked lesson distinguish three connection failures

TEXT

```
HTTP 401/403: application identity or permission was rejected
HTTP 426: ordinary HTTP reached a WebSocket-only path
HTTP 502: proxy could not obtain a valid upstream response
```

These are diagnostic categories, not an exhaustive mapping of every proxy implementation. Inspect browser network details, API logs, and Nginx error logs together. A normal `curl` request to a WebSocket endpoint does not perform the browser's authenticated upgrade handshake.

BASH

```
sudo nginx -t  
sudo journalctl -u nginx --since '10 minutes ago' --no-pager  
sudo tail -n 60 /var/log/nginx/error.log
```

Redact request URLs if they contain secrets before sharing output. Prefer webhook credentials in supported headers; if an external sender requires a query credential, minimize its exposure in access logs and rotate it under a controlled process.

Exercise: Make a browser test verify that an unauthorized WebSocket connection is rejected while an authorized one opens. Then stop the broker simulator and prove the dashboard remains available to report degraded broker status.

Chapter 34 Observability scheduling backups and cost

Build a dashboard of queue age, unknown orders, reconciliation lag, active positions lacking fresh prices, event-loop lag, broker error classes, Redis availability, disk usage, and descriptor count. Avoid labels containing arbitrary request IDs or every raw symbol on every metric; uncontrolled cardinality makes monitoring expensive. Use logs or traces for high-cardinality detail.

An SLI measures behavior, an SLO states a target, and an error budget describes allowed unreliability against that target. Separate market-session objectives from overnight periods. A feed-freshness objective should not declare an idle illiquid stock failed solely because it did not trade. Tie monitoring to provider heartbeat and subscription evidence too.

Useful read-only operations on a deployed instance include:

BASH

```
systemctl is-active ashuchart-api ashuchart-alert ashuchart-market-feed ashuchart-execution
ashuchart-reconciliation
sudo journalctl -u ashuchart-market-feed --since '10 minutes ago' --no-pager
sudo journalctl -u ashuchart-api --since '10 minutes ago' --no-pager
sudo journalctl --disk-usage
df -h /
free -h
sudo ss -ltnp
```

Collect logs in UTC when correlating services across machines; display IST separately. `journalctl -f` follows new lines and exits with Ctrl+C. The pager exits with q. Redirecting output with > writes to a file, so no terminal output is expected.

Set journal retention and application log rotation according to your incident and audit needs. Do not delete critical trading records merely to free space. Back up the ledger, configuration, and encrypted recovery material; verify restoration into a separate environment. A snapshot is not automatically application-consistent, and a backup is not useful until its restore procedure has been exercised.

Scheduling

An in-VM timer cannot start a stopped VM. Use Compute Engine instance schedules or an external scheduler for power operations. Google documents instance start/stop schedules and associated permissions. Scheduled times are not a real-time guarantee, so allow startup and reconciliation margin before accepting signals. [VM schedules](#)

A service-restart timer can use an explicit calendar timezone:

INI

```
[Timer]
OnCalendar=Mon..Fri *-*- * 09:00:00 Asia/Kolkata
Persistent=false
Unit=trading-school-restart.service
```

Validate with `systemd-analyze calendar 'Mon..Fri *-*- * 09:00:00 Asia/Kolkata'`. The referenced service must exist and implement a safe handover. `Persistent=true` can trigger a missed calendar event after boot, which may be undesirable for a trading restart. A weekday schedule does not encode exchange holidays. Do not stop a VM while it is responsible for unresolved orders or needed exit monitoring.

Cost model

Compute hours are only one line item. Add disks, snapshots, reserved/public IPv4 charges as applicable, network egress, logging, load balancers, managed databases, and taxes. A stopped VM can still incur charges for retained resources. Budget alerts notify; they are not an automatic spending cap. [Cloud Billing budgets](#)

Estimate monthly cost as `running_hours * compute_rate + provisioned_storage + IP/network + managed_services + observability`. Use the current regional pricing calculator, not a fixed rupee promise. An 08:00-16:00 weekday schedule is approximately eight hours times the actual weekdays, but broker authentication and preparation time still need planning. Measure the application before downsizing RAM.

Lab: Restore a database backup, compare row counts and fill totals, and document recovery time and data-loss window. Then inventory and remove your disposable resources explicitly. Stopping a VM is not cleanup of disks, reserved addresses, buckets, or endpoints.

Worked lesson write a restore acceptance test

A backup drill must specify what success means. For a trading ledger: all strategy versions referenced by open positions exist; execution IDs remain unique; total owned quantity matches the restored fill ledger; credentials can be recovered through the approved key process; and the restarted worker does not replay already-submitted orders blindly.

TEXT

```
Restore checkpoint
  record backup timestamp and schema version
  restore into an isolated database
  run integrity and quantity queries
  start paper workers with external network orders disabled
  replay duplicate broker evidence
  compare before and after totals
  record elapsed recovery time
```

Choose RPO and RTO from business requirements, then measure whether the backup and deployment process can meet them. A claimed five-minute RTO is not credible if restoring the database takes forty minutes.

Exercise: Produce a monthly cost worksheet with compute, persistent disk, snapshots, public IP, logging, database, and network entries. Set one workload to eight hours per weekday but keep persistent resources billed for their actual retention. Explain why scheduling is not a universal budget cap.

Chapter 35 Containers Terraform and managed runtime choices

A container packages a process and its filesystem dependencies, not a whole independent kernel. Build a non-root image, avoid secrets in layers, keep dependencies pinned, and scan the resulting artifact. Docker Compose is useful for a local API, Redis, PostgreSQL, and fake broker stack. It is not automatically high availability.

Terraform represents desired infrastructure state. Learn provider configuration, variables, resources, outputs, plans, state, and locking. Review a plan before apply. State can contain sensitive information; secure the backend and access controls. Import existing resources rather than creating duplicate production infrastructure accidentally. Prefer a disposable project to learn destroy operations.

Use GKE to learn Pods, Deployments, Services, ConfigMaps, Secrets, probes, resource requests/limits, rollout, and workload identity. A Kubernetes Secret is not automatically safe because it is base64-encoded. A Deployment with two replicas of an execution worker can double-order unless ownership is designed for it. Readiness probes should not cause a storm of broker calls.

Cloud Run can host appropriate stateless HTTP services and supports WebSockets, but connection timeout and multi-instance state synchronization still matter. Long-lived feed ownership and account-scoped order execution need an explicit design; do not assume a VM worker can be moved unchanged into a request-driven service. [Cloud Run WebSockets](#)

Learn service selection by workload:

Need	Candidate	Trade-off to explain
Long-running broker owner	Compute Engine or designed container worker	You manage supervision and recovery
Stateless authenticated HTTP	Cloud Run	Runtime lifecycle and external state
Many orchestrated workloads	GKE	Powerful controls, more operational concepts
Relational ledger	Cloud SQL PostgreSQL	Managed operations and ongoing cost
Ephemeral shared cache	Memorystore	Network access and durability requirements
Object archives	Cloud Storage	Object semantics, not row transactions
Analytics across large history	BigQuery	Query economics, not order-path OLTP

Lab: Containerize the paper API, then deploy only that API to a managed runtime. Keep its database external, document cold-start and connection-pool behavior, and test a rollout. Compare this with the VM deployment using measured complexity and cost, not fashion.

Worked lesson image versus container versus Pod

An image is a versioned filesystem and startup description. A container is a process instance created from it. A Pod is Kubernetes' scheduling unit that can contain one or more containers sharing networking and selected volumes. A Deployment describes how controllers maintain and replace Pods.

TEXT

```
Python source -> container image -> registry digest
                        |
                        Deployment specification
                        |
ReplicaSet -> Pod -> process
```

Use an image digest or a controlled immutable tag for releases. A mutable latest tag makes it harder to know what code is running. A restart is not a database migration, and scaling a Deployment does not make an embedded SQLite database shared.

Exercise: Follow the extended Kubernetes labs later in this edition. Explain which examples are intentionally single-replica and why. Do not enable an HPA for the SQLite lab simply because autoscaling appears in an exam objective.

Part Six Data Engineering Machine Learning and Certification

Chapter 36 Build an analytics path outside execution

Operational databases answer questions such as "is this position already exiting?" Analytics systems answer questions such as "what was the distribution of entry latency over six months?" Different access patterns justify different storage. Never make a live exit wait for a warehouse query.

Export sanitized domain events from the outbox into an analytics path. An event should contain a stable ID, schema version, event time, ingestion time, account scope, event type, and payload. Deduplicate on event identity. Separate event time from processing time so delayed arrivals are not assigned to the wrong market interval.

Start with daily Parquet files in Cloud Storage and a batch warehouse load. Add streaming only when the freshness requirement justifies it. Pub/Sub can decouple producers and consumers, but redelivery and acknowledgment behavior still require idempotent consumers. Dataflow is useful for managed Apache Beam pipelines with event-time windows and late-data handling. Dataproc is relevant when existing Spark/Hadoop workloads justify that environment. These are alternative tools, not mandatory steps on every event.

Partition BigQuery tables by an appropriate date and cluster by fields frequently used for selective filtering. Require partition filters where suitable and inspect bytes processed before running expensive queries. Small operational updates belong in the ledger, not a stream of warehouse polling requests. [BigQuery partitioning](#)

SQL

```
-- Illustrative BigQuery query after you create the analytics schema.
SELECT
  strategy_id,
  COUNT(*) AS submissions,
  APPROX_QUANTILES(submission_latency_ms, 100)[OFFSET(95)] AS p95_ms
FROM `learning_project.paper_events.submissions`
WHERE event_date BETWEEN DATE '2026-09-01' AND DATE '2026-09-07'
GROUP BY strategy_id;
```

Do not claim these dates contain real trading results. Generate synthetic data for the lab. Add schema checks, missing-value rules, expected row counts, lineage, access policy, and retention. If a source field changes its type, quarantine bad events rather than silently turning every price into zero.

Lab: Generate 10,000 paper fills and latency events, write a partitioned archive, load an analytics table, and verify duplicate imports do not double totals. Simulate one-hour-late data. Explain why an at-least-once pipeline needs business-key deduplication even if transport is reliable.

Worked lesson event schema evolution

JSON

```
{
  "event_id": "paper-fill-17",
  "schema_version": 2,
  "type": "fill_confirmed",
  "account_id": "synthetic-account",
  "event_time": "2026-09-18T04:00:00Z",
  "ingested_at": "2026-09-18T04:00:02Z",
  "quantity": 2,
  "price": "100.25"
}
```

Event time describes the business occurrence; ingestion time describes your observation pipeline. Late arrivals can change historical aggregates. Additive schema changes are often easier than changing the meaning of an existing field, but consumers still need validation and version policy.

Exercise: Add a fee field without breaking a version-one consumer. Then change price from a decimal string to a nested object and demonstrate why it is a breaking change. Quarantine invalid records with sanitized reasons and replay them after repair without duplicating accepted events.

Chapter 37 Statistics and honest backtesting

Before ML, learn mean, median, variance, standard deviation, quantiles, probability, conditional probability, correlation, sampling, and confidence intervals. Learn how a small number of outliers can make average latency or average trade return misleading. Correlation does not establish causation.

A backtest is a simulator using historical information. At decision time it must use only information that would actually have been available. A candle's final high is not known at its opening. If you choose symbols using today's surviving list, you can introduce survivorship bias. If you tune parameters repeatedly on the same test period, you have used that test period for training.

Use chronological train, validation, and test splits for time-dependent problems. Fit transformations only on training data. Where labels overlap across time, consider purging overlapping examples and an embargo around split boundaries. Compare with simple baselines before celebrating a complex model.

Include transaction costs, slippage assumptions, delayed entries, partial fills, circuit restrictions, and unavailable instruments in the simulator's limitations. Mark ambiguous intrabar outcomes. If both stop and target are inside the same OHLC candle, that candle alone does not reveal which happened first.

Track drawdown, turnover, exposure, distribution of returns, and sensitivity to costs rather than only win rate. A strategy can win often and lose money because occasional losses dominate. A promising backtest is not evidence that a system is safe to trade live or compliant with applicable rules.

Lab: Generate a synthetic trending and a synthetic random-walk price series. Implement a simple moving-average signal with next-bar execution. Then deliberately introduce look-ahead by using future data, observe the improvement, and explain why it is invalid. Repeat with higher costs and report the change honestly.

Worked lesson compute drawdown

PYTHON

```
equity = [100, 110, 105, 90, 108, 120]
peak = equity[0]
worst = 0.0
for value in equity:
    peak = max(peak, value)
    drawdown = (peak - value) / peak
    worst = max(worst, drawdown)
assert round(worst * 100, 2) == 18.18
```

This curve has a decline from 110 to 90 before reaching a new high. A positive final return does not reveal the depth of interim loss. Track both returns and risk measures, and state whether the curve includes fees and unrealized marks.

Exercise: Change execution from same-bar close to next-bar open and observe the difference. Include a missing candle and a gap. If target and stop both lie within one bar, implement a conservative ambiguity policy and compare results with finer-grained data where available.

Chapter 38 Machine learning and MLOps

ML estimates a mapping from features to outcomes. A training job minimizes a loss on examples; evaluation measures generalization on held-out data. Features must be available at prediction time. Labels must be defined before you choose a model. "Predict profitable trades" is not a complete label definition.

For a safer project extension, predict operational anomalies, such as unexpectedly delayed feed updates, using sanitized metrics. Keep predictions advisory. Another useful exercise is classifying sanitized error messages into categories for an operator, with confidence and an abstain option. Neither system needs broker trading permissions.

Learn regression versus classification, train/validation/test separation, overfitting, regularization, class imbalance, precision, recall, ROC/PR curves, and calibration. Accuracy is weak when almost every example is healthy. If missing a real outage is expensive, discuss false-negative cost explicitly rather than maximizing one generic score.

Start locally with a reproducible scikit-learn pipeline. Pin random seeds where applicable, record data versions and feature transformations, and compare against a constant or simple rule baseline. A seed does not guarantee bit-for-bit reproducibility across every platform and library version.

Map the workflow to Google's managed ML services: store data, run a training job, track an experiment, register an artifact, evaluate it, deploy batch or online inference, monitor, and retrain under controlled approval. Your attachment calls this Vertex AI; current documentation and exam guides may use updated Gemini Enterprise Agent Platform names for some capabilities. Follow the linked exam guide's terminology rather than assuming a service name remains unchanged.

MLOps applies software and data controls to models. Version code, data, features, parameters, and artifacts together. Training-serving skew occurs when the feature pipeline differs between training and inference. Data drift means the input distribution changed; concept drift means the relationship with the target changed. Neither automatically proves retraining will help.

Lab: Train an advisory latency-anomaly model, publish a model card with limitations, and compare batch predictions against a rule baseline. Deploy only in a disposable project, set resource limits, then remove the endpoint after the exercise. Real-time endpoints and accelerators can keep accruing cost even when you are not using the browser.

Worked lesson a leakage resistant baseline pipeline

PYTHON

```
# Requires scikit-learn in a separate ML learning environment.
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def fit_baseline(features, labels, split_index):
    model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))
    model.fit(features[:split_index], labels[:split_index])
    probabilities = model.predict_proba(features[split_index:])
    return model, probabilities
```

The scaler is fit only on the training slice because it is inside the fitted pipeline. This is only a basic chronological split; overlapping target windows and delayed feature availability can still leak information. Require both classes in training and validate the feature schema.

Exercise: Use synthetic operational metrics, not real client data, and compare the model with a simple threshold rule. Report precision and recall for the rare failure class. Add an abstain policy when required features are missing rather than filling them with invented zeros.

Chapter 39 Embeddings vector databases and retrieval

An embedding maps content to a numeric vector. Similarity search retrieves nearby vectors according to a metric. It does not prove factual correctness. A vector database stores embeddings and metadata and may index them approximately to trade exactness for speed and memory.

Use this technology for a read-only operations assistant over sanitized runbooks, architecture decisions, and known error explanations. Do not embed broker secrets, TOTP seeds, full customer identities, or unrestricted production logs. A retrieval system needs access control before returning results, not only before asking the language model to write an answer.

PYTHON

```
from math import sqrt

def cosine(a, b):
    if len(a) != len(b) or not a:
        raise ValueError("Vectors must have the same nonzero dimension")
    aa = sum(x * x for x in a)
    bb = sum(x * x for x in b)
    if aa == 0 or bb == 0:
        raise ValueError("Zero vector has no cosine direction")
    return sum(x * y for x, y in zip(a, b)) / sqrt(aa * bb)

assert abs(cosine([1, 0], [1, 0]) - 1) < 1e-12
```

This teaches the metric, not a scalable index. For production, use a tested library or service. Study exact search first, then HNSW and inverted-file approaches conceptually. Measure recall at k, latency, memory, and filtered retrieval behavior against an exact baseline.

Chunk documents by coherent meaning rather than arbitrary byte count. Store document ID, version, section, permissions, source URL, and timestamp. A change in embedding model or dimension usually requires a migration and re-embedding plan. A stale index can retrieve an old deployment instruction after the app changed.

RAG retrieves evidence and supplies it to a generator. Separate retrieval evaluation from answer evaluation. Test whether the right passage is retrieved, whether the answer is grounded, whether it cites the passage, and whether it declines unsupported claims. Retrieved text is untrusted data, not permission to execute instructions found inside it.

Compare a local PostgreSQL vector extension or other local engine with a managed service only after understanding the retrieval workload. Google's [Vector Search documentation](#) is the current product reference; product naming can change. Managed service selection should be based on scale, filtering, operations, and cost, not the mere presence of the word AI.

Lab: Index 20 sanitized incident notes, define 15 questions with expected source passages, and evaluate retrieval recall. Add a malicious instruction in a document and verify the assistant treats it as content. Give the assistant no broker, shell, secret-store, or production write credentials.

Worked lesson evaluate retrieval separately from generation

PYTHON

```
expected = {"q1": {"runbook-redis"}, "q2": {"runbook-ws", "runbook-nginx"}}
retrieved = {"q1": ["runbook-auth", "runbook-redis"], "q2": ["runbook-ws"]}
recalls = []
for question, relevant in expected.items():
    found = set(retrieved[question])
    recalls.append(len(found & relevant) / len(relevant))
assert sum(recalls) / len(recalls) == 0.75
```

This measures retrieval coverage for a tiny labeled example, not answer correctness. Evaluate whether a generated answer cites the retrieved evidence faithfully and distinguishes current from historical instructions. A correct-sounding answer with the wrong client account context is still a security failure.

Exercise: Add tenant metadata and a query from a different account. Verify filtering occurs before results reach the model. Test document deletion and re-indexing so removed sensitive text does not remain retrievable indefinitely.

Chapter 40 Choose a certification path

There is no single Google Cloud exam that certifies Python, software architecture, data engineering, ML, security, networking, and business leadership together. Use the attachment as a menu. For your goal, begin with engineering fundamentals and Associate Cloud Engineer, then choose Cloud Developer, Cloud Architect, or DevOps according to your work. Choose Data Engineer or ML Engineer after the data chapters and additional specialization practice.

Cloud Digital Leader and Generative AI Leader are useful for broad business understanding. Generative AI Leader is not a substitute for hands-on ML engineering or proof of production RAG expertise. Google's page explicitly targets business-level knowledge and does not require hands-on technical experience. [Generative AI Leader](#)

The official [Associate Cloud Engineer](#) page recommends at least six months of hands-on Google Cloud experience. The [Cloud Architect](#) page recommends broader industry and Google Cloud design experience. These are experience recommendations, not evidence that finishing a reading plan creates equivalent professional experience.

Do not treat AWS and Azure certifications as exact one-to-one equivalents. Vendor products, exam domains, prerequisites, and renewal rules differ. The Weaviate, Databricks, AWS, Azure, and course credentials mentioned in your attachment are optional separate paths; this book does not claim to prepare you fully for those vendor exams. Check their official current catalogs before paying for a course or booking an exam.

Associate Cloud Engineer preparation matrix

The currently linked standard guide groups the exam into environment setup, implementation, operations, and access/security, approximately 20%, 30%, 30%, and 20%. Do not rely on an older cached page with a different grouping. Use the current PDF linked from the certification landing page. [ACE standard guide](#)

Study block	Evidence to create	Extra practice beyond the trading VM
Environment setup	Project hierarchy, API and billing inventory	Organization policies, quotas, identity federation
Implementation	VM and container deployments with reviewed IaC	GKE, serverless events, alternative storage and networking
Operations	Restore drill, logging query, rollback report	Autoscaling, database backup tools, managed runtime operations
Access and security	Least-privilege service accounts and denied-access tests	Impersonation, workload identity, policy inheritance

Your VM project does not exercise every exam product. Maintain a gap sheet for products such as Spanner, Bigtable, Firestore, AlloyDB, shared networking, managed file storage, accelerators, and AI-assisted operations. For each, explain the use case, identity model, operational responsibility, and one reason not to choose it. Reconcile your gap sheet against every objective of the official guide before booking.

An eight week ACE revision cycle after the foundation

Week 1: resource hierarchy, projects, APIs, quota, billing, and budgets. Week 2: IAM, service accounts, impersonation, OS Login, and least privilege. Week 3: VPCs, routes, firewalls, DNS, load balancing, NAT, and private connectivity. Week 4: Compute Engine, disks, snapshots, managed instance groups, and scheduling. Week 5: containers, GKE basics, Cloud Run, and event triggers. Week 6: storage and database selection, backup, and restore. Week 7: monitoring, logs, troubleshooting, and cost investigation. Week 8: timed original practice questions, official sample questions, gap repair, and a fresh end-to-end lab without notes.

These weeks are revision blocks after practical study, not a promise that a beginner can skip experience. Mark each objective as explainable, labbed, and reviewed. Practice question percentages are your study feedback, not an official pass prediction.

Worked lesson build an exam gap tracker

CSV

```
objective,explain_without_notes,lab_evidence,review_date,next_action
IAM least privilege,yes,iam-denied-read.md,2026-09-23,repeat with workload identity
GKE probes,no,,2026-09-23,complete probe failure lab
Database restore,yes,restore-report.md,2026-09-23,repeat timed restore
```

The tracker records evidence rather than a vague feeling of readiness. Add every objective from the current guide for the exam you actually intend to take. Recheck the standard guide versus renewal guide; they can have different scope and format.

Exercise: Pick one incorrect practice answer and trace it to an objective, a misunderstood rule, and a small lab. Explain why the wrong choices fail the scenario constraints. Memorizing a product name without its constraints is weak preparation.

Chapter 41 Professional specialization labs

Cloud Architect

Prepare a design for 100 isolated client accounts, then a different design for one budget-constrained client. Explain availability targets, data residency, ownership, cost, recovery objectives, and operational staffing. Compare VM isolation with shared services and tenant-scoped authorization. Design migration from the existing Redis-centered system without losing open positions. Practice current official case studies rather than memorizing old case-study names. [Cloud Architect certification](#)

Deliver four diagrams, three architecture decision records, a cost model with dated assumptions, and a disaster-recovery plan. Defend a simpler design when it meets the requirements. You should be able to explain why more replicas can increase trading risk if account ownership is not coordinated.

Cloud Developer

Implement clean APIs, asynchronous jobs, secure service identity, configuration injection, structured errors, integration tests, and a controlled container rollout. Exercise a managed database connection pool under concurrency and show how authentication is preserved across HTTP and WebSocket paths. Review the current [Cloud Developer objectives](#) for products and skills your application does not cover.

Cloud DevOps Engineer

Create a CI/CD pipeline, artifact provenance record, SLO dashboard, error-budget policy, and two incident reports. Simulate a bad release and recover without duplicate paper orders. Study fleet operations, organization setup, delivery tooling, reliability, and observability beyond a single VM. The [DevOps exam guide](#) is the scope authority.

Cloud Security Engineer

Threat-model the admin setup page, webhook, broker callback, secret storage, CI pipeline, VM access, and backup archive. Design least privilege, incident evidence retention, key rotation, network controls, and separation of duties. Add a denied-access drill and investigate it with audit records. Follow the [Security Engineer guide](#); a password-and-TOTP implementation alone is far from its full scope.

Professional Data Engineer

Build batch and streaming ingestion of sanitized events, a governed analytical schema, lineage, late-data handling, data-quality checks, and a cost-aware dashboard. Exercise one backfill and one schema migration. Explain warehouse versus lake versus OLTP choices and how recovery preserves data fidelity. Review the [Data Engineer guide](#), including platform operation and governance beyond the example pipeline.

Professional Machine Learning Engineer

Implement reproducible training, evaluation, artifact registration, deployment, monitoring, and retraining for the advisory anomaly use case. Include responsible-AI considerations and explain why model output cannot authorize trades. Study managed ML, orchestration, scaling, and generative-AI evaluation against the current [ML Engineer certification page and linked guide](#). Coding the toy cosine function is not enough preparation for this role.

Database and Network specializations

For **Cloud Database Engineer**, investigate schema migration, indexing, query plans, replication, backup consistency, failover, and managed-database security. Prove a restore and compare isolation anomalies under concurrent writes.

For **Cloud Network Engineer**, build routing and firewall diagrams, troubleshoot private connectivity, compare load balancers, and study hybrid networking and DNS. A public VM with one firewall rule is only the beginning.

Workspace administration is a different specialization around an organization's collaboration environment. It is not necessary to add Workspace administration features to your trading app. Likewise, foundational leadership certifications can be studied conceptually without pretending they are prerequisites for every engineering certification. Check the [current certification catalog](#) because offerings and names change.

Worked lesson one problem three professional perspectives

Scenario: a client requires recovery after a zonal outage, restricted access to broker secrets, and audit history for all executions. The architect defines recovery goals, ownership, and a costed design. The DevOps engineer implements deployment, monitoring, failover drills, and safe handover. The security engineer reviews identities, secret access, evidence retention, and recovery abuse.

The database engineer tests backup consistency and ledger integrity. The network engineer investigates private connectivity and egress identity. The data engineer governs downstream analytics. The ML engineer ensures advisory models do not learn from leaked or unavailable-at-decision data. These roles overlap, but they are not identical exams or interchangeable job titles.

Exercise: Write one page from each of three perspectives, then identify a conflict. For example, broader log retention can help incident investigation but increase sensitive-data exposure and cost. Resolve it with scoped retention and redaction rather than pretending one requirement cancels the other.

Chapter 42 Original practice questions with explanations

These are original study questions, not exam dumps or official exam questions. Choose the best answer under the stated constraints, then explain why the alternatives are weaker.

1. A worker times out after submitting a buy. Should it retry immediately, mark rejected, or reconcile the durable intent? **Reconcile.** The broker may have accepted it; a transport timeout does not prove failure.
2. Browser /ws/feed returns 426, while broker health is connected. Which path do you inspect first? **Browser-proxy-API upgrade handling.** The outbound broker socket is separate.
3. The API needs one secret. Should its VM identity receive project Owner? **No.** Grant the required secret access at the narrowest practical scope and test denial elsewhere.
4. A monthly budget alert fired. Are resources automatically stopped? **No.** An alert is not a default hard spending cap. Investigate cost and apply an approved response.
5. A query scans six months when only one day is needed. What should you inspect? **Partitioning and filters, along with the query plan and bytes processed.** A larger VM does not fix warehouse query selection.
6. A managed database replica exists. Is backup restoration unnecessary? **No.** Replication can propagate deletion or corruption; recovery requirements still need backups and restore tests.
7. Two workers pass a duplicate-position check. What is missing? **Atomic reservation or equivalent ownership control.** Two sequential-looking Python checks are not one transaction.
8. A deployment uses two live execution replicas for availability. Is this automatically safe? **No.** Account ownership, fencing where possible, broker uncertainty, and idempotent recovery must be designed.
9. A model scores well after a random split of overlapping time-series samples. What is suspect? **Temporal leakage.** Use time-aware validation and examine overlapping label windows.
10. A vector search retrieves an old runbook. Does high cosine similarity prove the instruction is current? **No.** Version, provenance, permissions, and answer grounding matter.
11. A scheduled stop leaves a reserved IP and disk. Is the bill necessarily zero? **No.** Retained resources and other services may continue charging.
12. TOTP passes but the Dhan token expired. Can the app submit a broker order? **Not on that basis.** Application authentication and broker authorization are independent.
13. A Pub/Sub consumer processes a duplicate fill message. What prevents quantity doubling? **A durable execution-identity constraint and idempotent accounting.** Queue semantics alone are insufficient.
14. Redis PING returns AuthenticationError. Should you increase VM RAM? **No.** Verify the runtime credential source and server authentication settings first.
15. A Cloud Run instance has an in-memory WebSocket subscriber map. Will another instance share it? **No.** Use an external communication/state design and reconnect recovery.
16. A CNC holding is absent from one portfolio response. Should you delete local ownership? **No.** Investigate settlement phase, positions, response validity, and broker evidence.
17. The code handles a malformed SDK response by setting filled quantity to requested quantity. Is that resilient? **No.** It fabricates a fill. Preserve unknown state and reconcile.
18. A client requires every operation to stay available during a partition. Can you also guarantee one globally consistent execution owner without qualification? **Not generally.** State the partition behavior and prioritize safety for order submission.

19. An engineer proposes Kubernetes for a single low-volume client. What is the first question? **Which requirement or measured constraint does it solve?** Tool complexity is a cost.
20. All unit tests pass. Are production secrets, DNS, proxy upgrade headers, and provider permissions proven correct? **No.** Those require deployment and integration evidence at their own boundaries.

For every wrong answer, identify the misunderstood concept, repeat a small lab, and explain the repaired reasoning aloud. Read every objective in your chosen official exam guide; this question set cannot certify coverage or predict an exam outcome.

Additional scenario questions

Question: A Pod is Running but not Ready. Should you immediately increase replicas? **Answer:** Inspect readiness and dependency evidence first. More copies of the same misconfiguration will not repair it.

Question: An HPA has no CPU utilization value. What do you inspect? **Answer:** Metrics availability and resource requests, then HPA events. The target is calculated relative to requests for the relevant configuration, not an arbitrary machine percentage.

Question: A PVC survived Pod replacement. Is it a backup? **Answer:** No. It is persistent storage; deletion, corruption, and regional failure still need a backup and restore plan.

Question: An image push succeeded but a Pod reports ImagePullBackOff. What differs? **Answer:** The identity and network path pulling the image may differ from the deployer's push identity. Check reference, permissions, registry, and events.

Question: You enabled two replicas of a websocket owner. Is twice the feed availability guaranteed? **Answer:** No. You may exceed broker connection limits or create conflicting subscriptions. Partition ownership deliberately.

Exercise: Turn each answer into a reproducible paper-only failure drill and write the evidence that distinguishes it from a similar symptom.

Part Seven Your Rebuild Workbook

Chapter 43 The staged implementation contract

The reference lab is a small first vertical slice. It deliberately omits production login, distributed queues, live WebSockets, broker credentials, carry reconciliation, sector feeds, and cloud APIs. Those are your later assignments. Its ability to calculate a paper exit is not evidence that it can manage a brokerage account.

Create a new repository for your rebuild and keep the existing application read-only during study. Use one branch per milestone. At the start, write the behavior in plain English and write a failing test. At the end, explain the implementation without opening the code. Keep a short decision note recording what you intentionally did not build.

Milestone 1 A calculator you can trust

Implement strict capital sizing, explicit minimum-one policy, BUY/SELL P&L, target, stop, and tick alignment. Inputs must reject NaN, infinity, zero price, and unsupported sides. Deliver a CLI, ten or more meaningful boundary tests, and a one-page explanation of Decimal versus float. A function that silently invents defaults fails the milestone.

Milestone 2 Domain objects and a paper broker

Create signal, intent, order, fill, and position types. The broker accepts an order independently of filling it. Support delayed and partial fills in the simulator. Deliver a state diagram and prove that a repeated fill event has no second accounting effect. Explain why a broker ID is not an entry price.

Milestone 3 Durable configuration and accounting

Use SQLite locally to learn SQL, then PostgreSQL for concurrency exercises. Store strategy versions, signals, intents, and fills with constraints. Restart and recover records. Deliver migrations and a restore test. Do not store a client's actual broker token during this milestone.

Milestone 4 An HTTP vertical slice

Build create/list strategy, receive signal, list positions, and request exit routes. Define schemas and errors. Add duplicate-request semantics and conflict on reused IDs with different payloads. Deliver API tests that query the resulting database. Returning 200 without checking stored effects fails the milestone.

Milestone 5 Admin email password and TOTP

Implement restricted atomic setup, email ownership verification, password hashing, TOTP enrollment, full and pending sessions, CSRF, and recovery. Use a fake mail provider. Deliver concurrent setup and token redemption tests. Verify that a public visitor cannot seize the administrator account before its intended owner.

Milestone 6 A browser that reflects reality

Build configuration forms, open/closed positions, connection health, stale-price display, and understandable failures. Deliver Playwright tests at desktop and mobile sizes. Send an HTML 502 response to the frontend and confirm it shows a controlled error rather than a JavaScript crash.

Milestone 7 A deterministic risk engine

Add independent target, stop, trailing, cost-stop, pyramid, and alert-exit behavior. Preserve ownership by strategy and product. Deliver a tick replay with expected order intents and confirmed exits. A partially filled exit must not disappear from open positions until its remaining quantity is zero.

Milestone 8 Candle and sector filters

Add a historical-data interface, anchored breakout watches with TTL, exchange-calendar rules, and previous-session sector baselines. Deliver SHORT loser ranking tests, unknown-sector policy tests, and a watch-expiry race test. Replaying a watch after restart must not extend it.

Milestone 9 Queues and concurrent workers

Introduce durable signal processing and bounded workers. Add backpressure, admission control, poison-message handling, and a recovery policy. Deliver a crash-after-claim test and an unknown-submission test. Measure queue age under load. Do not replace uncertainty with blind retries to make tests easier.

Milestone 10 Broker adapters without live execution

Implement Dhan and Kite adapters using their installed SDK signatures and sanitized fixtures. Cover instrument mapping, quote normalization, market depth, order status, fills, candles, and holdings. Deliver a capability matrix and transport-level contract tests. Keep all test credentials fictional and all network order submission blocked.

Milestone 11 Feed ownership and reconciliation

Implement one connection owner, credential refresh, subscription replay, bounded tick delivery, source timestamps, and explicit degraded states. Reconcile orders, trades, positions, and holdings. Deliver a restart drill with carry positions and a lost order update. Report separately whether you proved connection, subscription request, or actual tick receipt.

Milestone 12 A cloud paper deployment

Deploy behind HTTPS using least privilege and protected secrets. Add health checks, monitoring, log retention, backups, and a measured restore. Deliver infrastructure definitions and a deployment runbook that another person can execute. The deployed application remains paper-only.

Milestone 13 Analytics and optional AI

Export sanitized events to an analytical store and add one advisory ML or RAG feature. Deliver leakage checks, retrieval/model evaluation, cost controls, and access restrictions. The AI component has no authority to submit, cancel, or modify a live order.

Milestone 14 Independent review

Ask another engineer to use the app, inspect one transaction race, attempt an unauthorized request in the test environment, and follow your recovery guide. Record findings, fix them, and add regressions. Senior-level growth includes responding to evidence and feedback, not merely producing more code.

Worked lesson definition of done for one milestone

For breakout monitoring, done means the saved configuration roundtrips; a closed candle is selected correctly; its threshold is persisted; TTL and entry-end boundaries are honored; duplicate signals do not create duplicate watches; a restart preserves the remaining deadline; one threshold crossing creates at most one intent; a kill switch blocks the entry; and the browser displays waiting, expired, and triggered states distinctly.

TEXT

```
Milestone evidence
specification.md
decision-record.md
tests for normal and boundary cases
one failure injection report
browser screenshot or trace where applicable
limitations.md
```

Exercise: Apply the same definition-of-done method to email verification, CNC restoration, and a cloud backup job. A feature's existence in a menu is not its acceptance criterion.

Chapter 44 Guided debugging cases and answer notes

Case A Configuration exists on screen but an alert says missing

Collect the incoming alert name, authorized account, canonical-name rule, stored configuration key, and worker environment. Compare exact strings before blaming Redis. A configured name `BULLISH_SCANNER` does not match incoming `high volume stock`. Also check that API and worker use the same intended Redis endpoint and database. Do not print credentials while comparing endpoints.

Expected reasoning: First separate name mismatch from tenant mismatch from storage/authentication failure. Add a correlation ID across intake and execution, and a sanitized reason explaining which lookup failed. Never fall back to an arbitrary strategy because its name seems similar.

Case B Price equals entry all afternoon

Check broker connection health, stock subscription identity, latest tick source, receipt time, queue delay, and browser connection. A `REST_EXIT_FALLBACK` price is not a WebSocket tick. A browser may also display stale data even while risk processing receives fresh data.

Expected reasoning: Trace the path hop by hop: broker packet, normalization, shared store/bus, risk consumer, dashboard publisher, browser DOM. Attach timestamps at each boundary. Do not assume a green service state proves the entire chain.

Case C A target appeared hit but no closed position appeared

Check whether the observed LTP was fresh and belonged to the right instrument; whether the configured exit applied to that product; whether one exit intent was reserved; whether the broker accepted and filled it; and whether the UI merged the final state correctly.

Expected reasoning: Trigger, request, acceptance, fill, ledger closure, and rendering are different facts. A UI status like `ENTERED_BUY` must not replace a confirmed closed-position record. Reconciliation and stable entity IDs should repair missed notifications.

Case D Too many open files after reconnecting

Measure the process descriptor count over repeated connect/disconnect cycles. Inspect event loops, HTTP sessions, sockets, thread lifetimes, and cleanup paths. Raising the limit can be temporary containment but does not establish that the leak stopped.

Expected reasoning: Reproduce in an isolated soak test. Assert descriptors and threads stabilize after many cycles. Close resources in `finally`, and make reconnect ownership singular. Do not create an event loop for each message.

Case E Admin email is allowed in a file but setup rejects it

Check which `EnvironmentFile` the running service loads, when it last restarted, any application `.env` loading precedence, address normalization, and the actual request email. Avoid displaying the entire process environment.

Expected reasoning: File contents are not automatically running process state. An updated file can coexist with an older in-memory value. Fix configuration precedence and provide a safe diagnostic that reveals only the allowed email policy, not secrets.

Case F Execution worker fails for roughly thirty seconds after restart

Examine whether the previous worker's ownership lease remains, whether it was released safely, and whether the new worker later recovers. Compare lock TTL and startup timing. Do not delete a lock merely to silence the error.

Expected reasoning: A lease may be intentionally protecting against overlap. Improve shutdown and startup observability, but preserve safety against a still-running old owner and uncertain broker submissions. Temporary lock contention is not proof Redis is slow.

Case G A market order was accepted but never filled

Distinguish pending, rejected, partial, cancelled, and unknown. Read broker-provided rejection details. Validate product, funds, security ID, tick size, circuit limits, and the actual serialized order type. A last-traded price is not executable ask depth.

Expected reasoning: Retry only under a bounded policy with fresh data and reconciled remaining quantity. Never retry an unknown outcome as though it were a confirmed rejection. Broker protection mechanisms and rules must be checked against the current provider contract.

Worked lesson evidence driven incident notes

TEXT

```
Observed: dashboard /ws/feed returned HTTP 426
Impact: browser did not receive live dashboard messages
Known: HTTP API routes returned responses
Unknown: whether Dhan ticks reached the market-feed process
Hypothesis: proxy did not forward upgrade headers
Next check: inspect active Nginx configuration and authenticated handshake
Containment: do not assume dashboard prices are fresh
```

Separate facts from inference. Later append the confirmed cause, fix, validation, and prevention. Avoid rewriting the original uncertainty as though you knew the cause at the beginning. This habit makes incident reports credible and prevents premature fixes to unrelated components.

Exercise: Write the same structure for `AuthenticationError`, `CFG_MISSING`, and `UNKNOWN_ORDER_OUTCOME`. Include a safe diagnostic command and a dangerous action you explicitly avoid, such as deleting Redis state or resubmitting blindly.

Chapter 45 Senior developer interview preparation

A senior engineer is expected to make trade-offs, debug across boundaries, communicate uncertainty, and improve how a team delivers software. Learning one project deeply can be powerful evidence, but years of operational judgment are not compressed into a certificate. Prepare to discuss decisions and failures honestly, including the role AI played in the original implementation.

A forty five minute system design exercise

Spend five minutes clarifying users, brokers, account isolation, volume, latency, market hours, and safety requirements. Spend five defining entities and invariants. Spend ten drawing the major components and data flow. Spend ten on order uncertainty, retries, reconciliation, and recovery. Spend five on scale and capacity. Spend five on security and operations. Use the final five to summarize trade-offs and unanswered risks.

State assumptions before choosing tools. Example: "For one client and a modest tick set, I start with supervised processes on one VM and a durable ledger. This minimizes cost but accepts a single-VM availability limitation. I would not call it highly available."

Questions you should answer without notes

1. Why is an accepted order not a position? Explain fill-based accounting and partial quantity.
2. How do two exit triggers avoid double-selling? Explain durable reservation and ownership.
3. What happens when a broker call times out? Explain unknown state and evidence-based recovery.
4. How does a cache differ from a ledger? Explain expiry, durability, and authoritative state.
5. When would you split a service? Explain workload isolation and operational cost.
6. How do you keep an event loop responsive? Explain bounded I/O offloading and CPU isolation.
7. How do you change an API safely? Explain versioned contracts and compatibility rollout.
8. Why can a unit test pass while the dashboard is broken? Explain proxy, browser, and deployment boundaries.
9. How do you secure first-admin setup? Explain out-of-band ownership and atomic initialization.
10. How do you recover after losing a database? Explain backups, reconciliation, RPO, and RTO.
11. How do you rank sectors correctly for short trades? Explain ascending valid changes and stale-data exclusion.
12. How would you use ML safely here? Explain an advisory boundary with no execution authority.

Use the structure requirement, alternatives, decision, evidence, consequence. Avoid claiming a feature is industry standard just because it uses Redis or Kubernetes. Show a test, a measurement, a diagram, or a failure analysis.

Portfolio evidence

Include a sanitized architecture document, a runnable paper demo, migration files, meaningful test results, one load report, a threat model, a restore report, and two incident postmortems. Record which code you wrote independently and which reference material you consulted. Never publish client credentials, brokerage screenshots with personal data, or private production logs.

Worked lesson an architecture decision record

TEXT

Decision: keep the broker feed owner on a supervised VM initially
Context: one client, modest subscription set, continuous outbound socket
Alternatives: VM worker, GKE worker, request-driven managed service
Chosen because: straightforward ownership and predictable process lifecycle
Costs: VM patching, single-instance limitation, explicit recovery work
Revisit when: measured scale or availability goals exceed this design
Evidence: reconnect soak test and operational runbook

This is stronger than saying a VM is always better than Kubernetes. An interview answer should be conditional on requirements and measured constraints. Explain which failure you accept, which you mitigate, and how you know the mitigation works.

Exercise: Present your design in ten minutes. Ask a peer to introduce one new constraint, such as 100 clients or regional outage recovery. Revise the design without abandoning established quantity and ownership invariants.

Chapter 46 Completion rubric and daily practice

Score each skill from 0 to 3. Zero means unfamiliar. One means you can follow instructions. Two means you can implement and explain independently. Three means you can diagnose failure, compare alternatives, and review another person's implementation. This is a personal learning rubric, not a hiring or certification standard.

Skill	Evidence for level 3
Core Python	Explain aliasing and exceptions; write tested pure functions
Advanced Python	Bound concurrency and correctly handle cancellation
API development	Validate, authorize, version, and observe a vertical slice
Database design	Enforce invariants under concurrent transactions
Authentication	Defend bootstrap, recovery, token redemption, and sessions
Broker integration	Handle partial fills and ambiguous side effects
Testing	Reproduce a race and prevent regression across browser/backend
Cloud operations	Deploy, diagnose, restore, and account for costs
System design	Defend a simpler and a scaled architecture with trade-offs
Data and ML	Detect leakage and isolate advisory models from execution
Communication	Write a clear incident report without overstating certainty

A productive daily session can be 20 minutes reviewing yesterday's concept, 50 minutes coding, 30 minutes testing, and 20 minutes explaining and recording decisions. Change the durations to fit your schedule. The important habit is producing your own reasoning, not consuming endless videos.

When using AI, first write your prediction and test. Ask for a hint, a counterexample, or a review rather than a full replacement. After reading help, close it and reimplement the concept. If you cannot explain the failure case that motivated the code, you have not yet learned that code.

Worked lesson self assessment through independent implementation

Choose a small feature, close the book, and implement it from a written specification. Run tests you prepared before coding. Explain the design to another person, then ask them to change a requirement. Being able to adapt is stronger evidence than reproducing memorized source.

Use this sequence for a monthly review: implement a pure rule; add a validated route; persist it; test concurrent access; demonstrate it in a browser; deploy it to a disposable environment; inject failure; recover; and write a short explanation. Repeat with a different feature so the exercise tests transferable skill.

Exercise: Maintain a learning journal with one incorrect assumption per week and the evidence that changed it. Examples include assuming every green status badge proves live data, assuming accepted equals filled, or assuming a budget alert stops billing. The goal is better judgment, not a claim that you never make mistakes.

Part Eight Extended Google Cloud and Kubernetes Labs

Chapter 47 Networking from addresses to application requests

The mental model

A network address identifies an interface within a routing context. A port identifies an application endpoint on that address. `127.0.0.1:8015` is loopback on the machine where the command runs, not automatically your VM. A browser on your laptop reaches the VM only through a routable endpoint, an authenticated tunnel, or a proxy.

TCP establishes a reliable byte stream; it does not understand whether bytes represent a valid order. TLS protects the connection and verifies the server identity when configured correctly. HTTP adds request semantics. WebSocket upgrades an HTTP handshake into a bidirectional message channel. Each layer can succeed while a higher layer fails.

Use CIDR notation to describe a network prefix. For `10.42.0.0/24`, 24 bits are the network prefix. Cloud providers reserve some addresses in each subnet, so do not assume every mathematically available address can be assigned to a VM.

PYTHON

```
from ipaddress import ip_network, ip_address

subnet = ip_network("10.42.0.0/24")
assert ip_address("10.42.0.8") in subnet
assert ip_address("10.43.0.8") not in subnet
assert subnet.num_addresses == 256
```

Trace a private database connection

Suppose the API has a private address and Cloud SQL is reached privately. You need identity permission, a valid database credential or IAM database identity, a reachable network path, compatible connection settings, and database authorization. The Cloud SQL Auth Proxy authenticates and encrypts its connection but does not magically create missing private network reachability. Distinguish IAM permission to connect from SQL permission to select a table.

Diagnose in layers

Start with the exact destination and port. Resolve DNS. Inspect the listening process. Check route and firewall policy. Check TLS hostnames. Then inspect application authentication and response data. Do not disable every firewall rule to determine whether a login password is wrong.

BASH

```
# On your learning VM, read-only diagnostics:
hostname
ip address
ip route
ss -ltn
curl --max-time 5 -i http://127.0.0.1:8015/health
```

Expected response from the paper lab contains `PAPER_ONLY`. A timeout, connection refusal, 401, and 502 are different evidence. Write down the observed category before changing configuration.

Exercise and interview checkpoint

Draw laptop, IAP tunnel, VM, Nginx, API, and database. Label private and public addresses, ports, TLS termination, and identity checks. Explain how a request reaches a private VM without exposing SSH globally. Explain why allowing ingress port 443 does not grant access to the broker account.

Chapter 48 Containerize the paper application

Prerequisites and scope

Install Docker using its official platform instructions, use Linux containers, and confirm `docker version` shows both client and server. On Windows, the local runtime may use WSL2. Do not mix Windows container images with the Linux example. Docker is not installed in the authoring environment, so the following build is a lab you must run; it is not reported as already executed.

Build from `docs/learning-book/lab`. The Dockerfile copies only the learning package and runtime dependencies. It does not copy the production app or its credentials.

DOCKERFILE

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PAPERLAB_DB=/data/paperlab.sqlite3
WORKDIR /app
COPY requirements-runtime.txt ./
RUN python -m pip install --no-cache-dir -r requirements-runtime.txt \
    && groupadd --gid 10001 paper \
    && useradd --uid 10001 --gid paper --no-create-home paper \
    && mkdir /data \
    && chown paper:paper /data
COPY --chown=paper:paper paperlab ./paperlab
USER 10001:10001
EXPOSE 8015
CMD ["python", "-m", "uvicorn", "paperlab.api:default_app", "--factory", "--host", "0.0.0.0", "--port", "8015"]
```

Read the Dockerfile line by line

The base image supplies Python and Linux userspace. `WORKDIR` sets the process working directory. Copying the small dependency file before application source allows unchanged dependencies to reuse a cached build layer. A non-root user limits some consequences of compromise. `EXPOSE` is metadata; it does not publish a port by itself.

The JSON-array `CMD` launches Python directly, improving signal delivery compared with hiding the process behind an unnecessary shell. Binding Uvicorn to `0.0.0.0` inside the container makes it reachable through container networking. The host publishing rule below still binds only to laptop loopback.

BASH

```
docker build -t paper-school:lesson1 .
docker run --rm --name paper-school-local \
    -p 127.0.0.1:8015:8015 paper-school:lesson1
```

Open `http://127.0.0.1:8015`, save a strategy, and send a synthetic signal as described in the lab README. In another terminal:

BASH

```
docker logs --tail 50 paper-school-local
docker stats --no-stream paper-school-local
docker inspect paper-school-local --format '{{.Config.User}}'
```

Expected configured user is `10001:10001`. The database currently lives in the container's writable storage. Removing this container removes that synthetic data. Do not interpret this as a production persistence design.

Reproducibility and security

The image tag `python:3.12-slim` can move. For a release, record and review a digest, rebuild for security updates, scan dependencies, and run tests. The teaching dependency versions mirror the small lab; they are not a certification of current production security. Never `COPY .env` into an image and then delete it in a later layer, because earlier layers can retain the content.

Exercise

Stop the local container with `docker stop paper-school-local`. Run it again and check which records remain. Then mount a named volume at `/data`, create another record, and repeat. Explain the difference between process lifetime, container filesystem lifetime, and named-volume lifetime. Remove only the named teaching volume after confirming the synthetic data is disposable.

Chapter 49 What Kubernetes actually controls

Kubernetes is a desired-state control system. You submit objects to an API server. Controllers compare desired and observed state and take action. The scheduler chooses suitable nodes for unscheduled Pods; the kubelet manages containers on a node. Cluster state is maintained through the control plane. Kubernetes does not understand whether a broker order is duplicated or whether a strategy's target was reached. [Kubernetes components](#)

TEXT

```
kubectl -> API server -> persisted object state
                        |
                        controllers observe difference
                        |
                        scheduler selects a node
                        |
                        kubelet starts container
                        |
                        status returns to API server
```

Objects and reconciliation

A manifest contains apiVersion, kind, metadata, and usually spec. Status is observed state; do not put a fabricated healthy status into a manifest. Labels are key-value selectors used to associate objects. An annotation carries metadata that is not intended as a selector.

Namespaces organize names and policy scope. They are not complete security isolation by themselves. Two client namespaces still require network controls, RBAC, resource limits, and data/secret separation. If administrators share unrestricted cluster access, they may see both clients.

Failure thought experiment

When a container exits, the Pod's restart policy may restart it. When a Pod disappears, its controlling workload may create a replacement. When a node fails, recovery depends on the cluster and storage design. None of these actions repairs a corrupt ledger or determines whether an order was accepted before the crash.

Exercise: Write two columns: orchestration responsibility and application responsibility. Put replacing a failed API Pod in the first, and reconciling an unknown order in the second. Place database failover and secret rotation carefully: both involve infrastructure and application behavior.

Chapter 50 Your first local cluster with kind

Use kind to run a local Kubernetes learning cluster in containers. Install a supported Docker runtime, kind, and kubectl from their official instructions, then confirm version compatibility. The [kind quick start](#) documents cluster creation and loading local images. Do not use this cluster to hold real broker credentials.

Create and inspect

From the paper lab directory, build the image first, then run:

BASH

```
kind create cluster --name paper-school
kubectl --context kind-paper-school cluster-info
kubectl --context kind-paper-school get nodes
kind load docker-image paper-school:lesson1 --name paper-school
kubectl --context kind-paper-school apply -f k8s/namespace.yaml
```

Using `--context` explicitly reduces accidental changes to a client cluster. Always inspect `kubectl config current-context` before destructive operations. Your kubeconfig can contain access credentials; treat it as sensitive.

The namespace manifest is:

YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: paper-school
  labels:
    purpose: learning-only
    pod-security.kubernetes.io/enforce: restricted
```

Apply and understand the output

created means the API accepted a new object. configured means an existing object was updated. Neither means the image pulled, container started, or application is ready. Use workload status and events for those questions.

BASH

```
kubectl --context kind-paper-school apply -f k8s/paper.yaml
kubectl --context kind-paper-school -n paper-school get pods -o wide
kubectl --context kind-paper-school -n paper-school get events \
  --sort-by=.metadata.creationTimestamp
```

Cleanup boundary

When the exercises are finished and synthetic data is no longer needed, `kind delete cluster --name paper-school` removes that named local cluster. It does not remove every Docker image on your computer. Do not use broad Docker prune commands against a machine that also hosts unrelated work.

Exercise: Intentionally use an image tag that does not exist. Inspect events and describe the difference between object creation and successful execution. Repair the image reference, then wait for rollout readiness.

Chapter 51 Deployments labels and a complete paper manifest

A Deployment manages a changing set of Pods through ReplicaSets. Its selector must match its Pod-template labels. A Service uses its own selector to find Pods. A typo between those labels can produce a healthy-looking Pod with no reachable Service endpoints. [Deployment reference](#)

The supplied lab deliberately uses one replica and Recreate. It stores a disposable SQLite file in `emptyDir`. This avoids presenting multiple uncoordinated SQLite instances as a scalable system. Pod replacement loses the synthetic database. Production durability is a later design, not a hidden property of this example.

YAML

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: paper-api
  namespace: paper-school
automountServiceAccountToken: false
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: paper-settings
  namespace: paper-school
data:
  PAPERLAB_DB: /data/paperlab.sqlite3
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: paper-api
  namespace: paper-school
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: paper-api
  template:
    metadata:
      labels:
        app: paper-api
    spec:
      serviceAccountName: paper-api
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: api
          image: paper-school:lesson1
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8015
          envFrom:
            - configMapRef:
                name: paper-settings
          resources:
```

```

    requests:
      cpu: 250m
      memory: 256Mi
      ephemeral-storage: 256Mi
    limits:
      cpu: "1"
      memory: 512Mi
      ephemeral-storage: 1Gi
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
    startupProbe:
      httpGet:
        path: /health
        port: http
      periodSeconds: 2
      failureThreshold: 30
    readinessProbe:
      httpGet:
        path: /health
        port: http
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /health
        port: http
      periodSeconds: 10
      failureThreshold: 3
    volumeMounts:
      - name: data
        mountPath: /data
      - name: temporary
        mountPath: /tmp
  volumes:
    - name: data
      emptyDir: {}
    - name: temporary
      emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: paper-api
  namespace: paper-school
spec:
  type: ClusterIP
  selector:
    app: paper-api
  ports:
    - name: http
      port: 80
      targetPort: http

```

Understand each group

The ServiceAccount names the Kubernetes workload identity and disables automatic API-token mounting because the lab does not call Kubernetes. The ConfigMap supplies a nonsecret database path. The Pod security context uses non-root UID and group values, an appropriate seccomp profile, and a group for mounted-volume access. The container drops capabilities and uses a read-only image filesystem; /data and /tmp remain writable mounts.

Requests describe scheduling needs. Limits constrain usage. The probe paths refer to the lab's actual /health endpoint. The Service remains ClusterIP, so no public load balancer is created for an unauthenticated learning application.

Verification

BASH

```
kubectl --context kind-paper-school -n paper-school rollout status \  
  deployment/paper-api --timeout=180s  
kubectl --context kind-paper-school -n paper-school get deployment,replicaset,pod,service  
kubectl --context kind-paper-school -n paper-school logs deployment/paper-api --tail=60
```

Exercise: Change a harmless annotation on the Deployment itself, then on the Pod template, and observe which causes replacement. Predict the result before running. Do not scale this SQLite exercise above one replica; first implement an external shared ledger and test concurrency.

Chapter 52 Resource requests limits and health probes

A request helps the scheduler place a workload. A CPU limit can cause throttling; a memory limit can lead to termination when exceeded. Increasing a memory limit does not repair an unbounded queue. Record actual working set, event-loop lag, and peak load before choosing values.

In the lab, 250m means one quarter of a CPU unit requested. 256Mi is a memory request, distinct from decimal megabytes. The example values are starting points for measurement, not capacity guarantees for the real trading app. Autopilot or admission policy may adjust resource settings; inspect the admitted Pod.

Three probe questions

Startup asks whether initialization has completed. Readiness asks whether traffic should be sent to this Pod. Liveness asks whether restarting the process is warranted. A startup probe can protect slow initialization from premature liveness failure. Failed readiness removes normal Service traffic eligibility but does not by itself restart the process. [Probe documentation](#)

The small lab uses /health for all three because it has no live provider dependency. A production rebuild should distinguish them. A disconnected broker should not automatically cause the dashboard process to restart, and a database outage should not cause every Pod to enter a synchronized restart loop.

Commands and interpretation

BASH

```
kubectl -n paper-school describe pod POD_NAME
kubectl -n paper-school logs POD_NAME --previous
kubectl -n paper-school top pods
```

Replace POD_NAME with an actual name and verify your context first. --previous inspects the previous container instance after a restart. top requires a working metrics pipeline, which a basic local cluster may not include. Missing metrics is not the same as zero usage.

Exercise: Use a separate test deployment with a wrong readiness path and observe Running but not Ready. Then repair it. Inject slow startup and tune startup-probe allowance. Document why liveness should not make a fresh broker order call on every probe.

Chapter 53 Services DNS and private browser access

Pod addresses change. A Service provides a stable discovery abstraction for matching workloads. ClusterIP is internal; NodePort and LoadBalancer expose different paths with different infrastructure implications. A Service does not create application authentication. [Service documentation](#)

Open the paper lab safely

BASH

```
kubectl --context kind-paper-school -n paper-school port-forward \
  service/paper-api 8015:80
```

Open <http://127.0.0.1:8015>. Keep the port-forward process running while using the browser. Ctrl+C stops the tunnel, not the Pod. Port-forward is an operator debugging path and is not a test of ordinary Service load-balancing behavior or network-policy enforcement.

Inspect the Service and EndpointSlices:

BASH

```
kubectl --context kind-paper-school -n paper-school get service paper-api -o yaml
kubectl --context kind-paper-school -n paper-school get endpointslices \
  -l kubernetes.io/service-name=paper-api
```

No ready endpoint can indicate selector mismatch or readiness failure. A DNS name resolving to a Service address does not prove there is a ready backend. Inside the namespace, `paper-api` is a useful short service name; cross-namespace access requires the intended namespace-qualified name under cluster DNS conventions.

Public traffic is a later exercise

Ingress and Gateway resources require compatible controllers. Merely applying an Ingress object does not install a controller or a certificate. Before exposing your rebuild, add tested authentication, authorization, HTTPS, request limits, session policy, and WebSocket support for the chosen controller. Do not expose the supplied unauthenticated lab through LoadBalancer.

Exercise: In an isolated cluster, change the Service selector to a nonexistent label. Observe Service existence with no matching backends. Restore it and explain why restarting the application was unnecessary.

Chapter 54 Configuration secrets RBAC and network policy

Use ConfigMaps for nonsecret settings. Use a controlled secret mechanism for credentials. Kubernetes Secret data commonly appears base64-encoded in manifests; encoding is not encryption. Restrict RBAC, enable appropriate at-rest protection, and avoid committing secret manifests to Git. [Kubernetes Secret practices](#)

Environment variables loaded at container start do not automatically update when a ConfigMap changes. Mounted configuration has different update behavior, but applications must still reload it safely. Broker-token rotation needs a versioned, explicit handover; it is not solved by editing one Kubernetes object.

Minimal RBAC lesson

The paper API does not need Kubernetes API permissions. For a separate observer service, a namespaced Role can permit only reading Pod status:

YAML

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-observer
  namespace: paper-school
rules:
- apiGroups: [""]
  resources: [pods]
  verbs: [get, list, watch]
```

A RoleBinding must attach it to the intended identity before it grants access. Do not bind it to every service account. `kubectl auth can-i` can help test permissions, but impersonation checks require privileges of their own. Reading Secrets is a different permission; do not grant it to a log viewer by habit.

NetworkPolicy lesson

YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: paper-api-isolated
  namespace: paper-school
spec:
  podSelector:
    matchLabels:
      app: paper-api
  policyTypes: [Ingress, Egress]
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: paper-test-client
    ports:
    - protocol: TCP
      port: 8015
  egress: []
```

This selects the paper API, allows ingress only from labeled test-client Pods in the same namespace on 8015, and allows no egress. It is optional and requires a network implementation that actually enforces policy.

Default kind networking may not. The paper app has no external dependency, so denying egress is an intentional lesson. A real broker worker would need carefully designed DNS and broker/Google API egress.

[NetworkPolicy semantics](#)

Exercise: Use a policy-enforcing cluster to compare two test Pods, one carrying the allowed label and one without it. Verify allowed and denied communication through the normal Service path, not just port-forward. Record the CNI and policy behavior used by the test.

Chapter 55 Persistent storage and database placement

`emptyDir` belongs to a Pod's lifetime. Container restarts within that Pod can preserve it, while Pod replacement removes it. A persistent volume claim asks for storage independent of one Pod. Its storage class, access modes, reclaim policy, and topology affect actual behavior. A PVC is not a backup. [Persistent volumes](#)

A teaching claim

YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: paper-data
  namespace: paper-school
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
```

This assumes a default storage class and compatible provisioner; otherwise specify a supported class or expect Pending. To use it, replace only the data volume's `emptyDir` with `persistentVolumeClaim: {claimName: paper-data}` in a separate teaching manifest. Keep one replica. `ReadWriteOnce` generally describes node attachment access, not an application-level single-writer guarantee across every possible Pod arrangement.

Why not put every database in Kubernetes immediately

A `StatefulSet` provides stable identities and storage relationships; it does not implement database replication, backup consistency, schema migration, or failover correctness for you. Self-managed PostgreSQL and Redis require operational expertise. A managed database may be preferable when its cost and constraints fit.

For the next rebuild stage, place the authoritative ledger in PostgreSQL and keep API Pods stateless with bounded connection pools. Let Redis hold cache and coordination data under a defined persistence policy. Keep schema migration as a controlled deployment step, not a race executed independently by every Pod at startup.

Exercise: Create a paper record, restart a container, replace a Pod, and compare outcomes with `emptyDir` and with a PVC. Back up the database, delete a test record, restore into an isolated instance, and verify totals. Explain the difference between persistence, replication, and recovery.

Chapter 56 Rollouts autoscaling jobs and safe ownership

Stateless HTTP services can often use rolling replacement. Your execution owner is different: overlapping old and new workers can be unsafe without account ownership and reconciliation. The paper SQLite lab uses Recreate and remains single-replica. Even Recreate is not a substitute for an application ownership protocol in the real system.

BASH

```
kubectl -n paper-school rollout history deployment/paper-api
kubectl -n paper-school rollout status deployment/paper-api --timeout=180s
kubectl -n paper-school rollout undo deployment/paper-api
```

Use these only after checking context and whether the earlier revision and its data assumptions are valid. Code rollback does not undo broker side effects or reverse a destructive migration.

HPA as a separate stateless exercise

YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stateless-dashboard
  namespace: paper-school
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stateless-dashboard
  minReplicas: 2
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

This refers to a future `stateless-dashboard` deployment you must implement with external state and CPU requests. It is not applicable to the supplied SQLite lab. HPA needs a metrics source; CPU utilization targets depend on requests. Autoscaling cannot fix a provider's fixed account rate limit. [HPA documentation](#)

Jobs and schedules

A Job runs finite work; a CronJob creates scheduled Jobs. Use a migration Job with explicit approval, a bounded retry policy, and idempotent logic. For a morning task, specify `timeZone: Asia/Kolkata` on a supported cluster and define missed-run behavior. `concurrencyPolicy: Forbid` reduces overlap for that CronJob but does not make the task globally exactly once. [CronJob documentation](#)

Exercise: Run a synthetic cache-refresh Job twice and prove its idempotent result. Then schedule it, suspend the CronJob, and inspect its state. Explain why a cluster CronJob cannot start a completely stopped cluster or VM on which it depends.

Chapter 57 Deploy the paper image to GKE

Cost and prerequisites

This is an optional billable exercise. Complete the local cluster first. Use a disposable project with billing, budget alerts, quota, and administrator-approved IAM. GKE can exceed a small single-VM monthly budget; the book does not promise a rupee ceiling. Read the current [Autopilot creation guide](#) and regional pricing before proceeding.

Run in Cloud Shell or a workstation with gcloud, Docker, kubectl, and the GKE authentication plugin available. Confirm versions and authorized project. The operator needs permissions for enabling APIs, creating the repository/cluster, building or pushing the image, and accessing the cluster. Do not solve missing permissions by granting Owner indiscriminately.

BASH

```
export PROJECT_ID="REPLACE_WITH_DISPOSABLE_PROJECT"
export REGION="asia-south1"
export CLUSTER="paper-school-gke"
test "$PROJECT_ID" != "REPLACE_WITH_DISPOSABLE_PROJECT" || exit 1
gcloud config set project "$PROJECT_ID"
gcloud services enable container.googleapis.com \
  artifactregistry.googleapis.com --project="$PROJECT_ID"

gcloud artifacts repositories create paper-school \
  --repository-format=docker --location="$REGION" \
  --project="$PROJECT_ID"

gcloud auth configure-docker "$REGION-docker.pkg.dev"
export IMAGE="$REGION-docker.pkg.dev/$PROJECT_ID/paper-school/paper-api:lesson1"
docker build -t "$IMAGE" .
docker push "$IMAGE"
```

The working directory must be the paper lab containing the Dockerfile. Authentication for pushing an image is not automatically authentication for nodes pulling it. Grant the cluster's actual image-pull identity repository reader access when required, and inspect ImagePullBackOff events instead of assuming the deployer's credentials are reused. [Artifact Registry image operations](#)

Create and access the cluster

BASH

```
gcloud container clusters create-auto "$CLUSTER" \
  --region="$REGION" --project="$PROJECT_ID"
gcloud container clusters get-credentials "$CLUSTER" \
  --region="$REGION" --project="$PROJECT_ID"
kubectl config current-context
kubectl apply -f k8s/namespace.yaml
```

For this exercise, create paper-gke.yaml from k8s/paper.yaml and replace its image: paper-school:lesson1 with the exact Artifact Registry image you just pushed. Keep the other safety properties. Inspect the resulting file before applying it:

BASH

```
kubectl apply -f paper-gke.yaml
kubectl -n paper-school rollout status deployment/paper-api --timeout=300s
kubectl -n paper-school get pods -o wide
kubectl -n paper-school port-forward service/paper-api 8015:80
```

Do not deploy the unresolved local-only image name to GKE. After deployment, inspect the actual Pod image reference and events. A registry image exists independently of your local kind image cache.

Keep access through the local port-forward or Cloud Shell's authorized preview. Do not create a public LoadBalancer for this lab. Test health, strategy save, one synthetic entry, and one synthetic target exit. Record admitted resource settings and workload events.

Cleanup

After verifying the project/context and deciding the synthetic data is disposable:

BASH

```
gcloud container clusters delete "$CLUSTER" \
  --region="$REGION" --project="$PROJECT_ID"
gcloud artifacts repositories delete paper-school \
  --location="$REGION" --project="$PROJECT_ID"
```

These commands are destructive to the named learning resources and prompt for confirmation. Review any retained disks, addresses, log retention, and other resources separately. Deleting a cluster is not proof that every related charge has stopped.

Chapter 58 Workload identity Cloud SQL and delivery pipelines

Workload identity instead of key files

Workload Identity Federation for GKE lets workloads use Google Cloud authorization without baking service-account keys into images. Kubernetes service accounts and Google Cloud service accounts are distinct identities. Follow the current [GKE workload identity guide](#) for the chosen direct-principal or impersonation approach.

The following direct-principal example grants one service account access to one existing synthetic secret. It assumes an appropriately configured GKE workload identity pool and a Kubernetes service account named `paper-api` in `paper-school`. The base paper image does not read Secret Manager, so this is an identity exercise for an extended test workload, not a required permission for the original lab.

BASH

```
export PROJECT_NUMBER="$(gcloud projects describe "$PROJECT_ID" \
  --format='value(projectNumber)')"
export PRINCIPAL="principal://iam.googleapis.com/projects/$PROJECT_NUMBER/locations/global/workloadIdentityPools/$PROJECT_ID.svc.id.goog/subject/ns/paper-school/sa/paper-api"

gcloud secrets add-iam-policy-binding paper-demo-secret \
  --project="$PROJECT_ID" \
  --member="$PRINCIPAL" \
  --role=roles/secretmanager.secretAccessor
```

Create only a harmless demonstration secret beforehand, enable Secret Manager, and use a reviewed Google client library in a separate test Pod. Validate allowed access and denial to a different secret. Do not print the retrieved value into shared logs; report only the outcome. Understand the cluster's metadata access and policy requirements before adding a deny-all egress policy.

Cloud SQL connection reasoning

The Cloud SQL Auth Proxy or supported connectors can simplify authenticated encrypted connections. They still require instance connectivity, IAM permissions, and database authorization. A proxy process failing to connect does not mean a SQL password is wrong; distinguish infrastructure authentication, network reachability, and database login. [Cloud SQL Auth Proxy](#)

Store a durable ledger in Cloud SQL only after implementing migrations and bounded pools. Estimate total possible connections as maximum Pods times pool size plus background and administrative clients. An autoscaling API can exhaust a database even when each Pod seems modest. Use graceful pool shutdown and retry only safe transactional operations.

CI delivery without broker credentials

Separate build identity, deploy identity, and runtime identity. The build runs tests and creates an immutable image. The deployer updates a reviewed workload specification. The runtime reads only its scoped resources. Untrusted pull requests must not access client secrets or deploy to production. Use workload federation for CI where supported rather than long-lived downloaded keys.

Exercise: Write a pipeline diagram showing where source, dependencies, test artifacts, image digest, approvals, and deployment identity enter. Add a failure before rollout and another after partial rollout. Explain the rollback behavior without assuming a database migration can always be undone.

Chapter 59 Infrastructure as code observability and cost control

Terraform as a reviewed plan

This small example creates only a network and subnet in a disposable project. Install Terraform and a supported Google provider, authenticate through an approved method, then review the plan. It does not deploy the trading application.

HCL

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
    }
  }
}

variable "project_id" { type = string }

provider "google" {
  project = var.project_id
  region  = "asia-south1"
}

resource "google_compute_network" "school" {
  name                  = "paper-iac-network"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "school" {
  name          = "paper-iac-subnet"
  ip_cidr_range = "10.77.0.0/24"
  region        = "asia-south1"
  network       = google_compute_network.school.id
}
```

After your first reviewed initialization, pin a compatible provider constraint and commit the dependency lock file. Protect state; do not commit local state files containing sensitive resource information. Configure an appropriate protected remote backend and locking before team use. Read the plan for replacements, not merely the number of changed resources.

BASH

```
terraform init
terraform fmt -check
terraform validate
terraform plan -var="project_id=$PROJECT_ID" -out=paper.plan
terraform show paper.plan
terraform apply paper.plan
```

The apply command changes cloud resources. Run it only after confirming the learning project and reviewing the plan. At the end, review `terraform plan -destroy` before approving destruction of these named learning resources. Do not run a destroy from a production state directory.

Operations evidence

Capture deployment version, Pod restarts, unknown orders, reconciliation lag, queue oldest age, active-position freshness, and user-visible request errors. Cluster CPU is not a trading correctness metric. Logs need account-safe correlation but should avoid raw secrets and uncontrolled high-cardinality labels.

TEXT

```
resource.type="k8s_container"  
resource.labels.namespace_name="paper-school"  
severity>=ERROR
```

This Cloud Logging filter is an exercise starting point. Plain stdout may not map to the severity you expect unless logging is structured appropriately. Search raw message text and inspect sample entries when a severity filter returns nothing.

Cost and cleanup checklist

Inventory clusters, node/Pod resource billing, Cloud SQL instances, disks/PVC backing volumes, snapshots, external load balancers, NAT, public addresses, Artifact Registry storage, and logs. Set budget alerts, but remember they are not automatic hard caps. A PVC's reclaim policy affects whether backing storage remains. Verify deletion rather than assuming a removed namespace removed every cloud resource.

Exercise: Produce a before/after resource inventory and a cost estimate with the date and regional assumptions. Explain why a small VM may remain the better production choice for one client even after you learn Kubernetes thoroughly.

Chapter 60 A full paper system capstone

Required architecture

Build an authenticated API with external PostgreSQL, Redis queues, one account-scoped execution owner, a simulated market stream, reconciliation, and a browser dashboard. Package separate processes in containers. Deploy locally first and to a disposable cloud environment only after tests pass. No real broker token is needed.

The deployment should make ownership explicit: API replicas may scale after session and state externalization; strategy evaluation may scale within provider-style quotas; market feeds are partitioned by account; execution is serialized or fenced per account under a documented protocol; reconciliation deduplicates evidence; notifications are independent.

Acceptance scenario

1. Provision infrastructure and schema from reviewed definitions.
2. Bootstrap one admin, verify email through a fake inbox, enroll TOTP, and sign in.
3. Save two strategies with different exit alerts and zero retries on one.
4. Deliver a duplicate signal and prove one intent is created.
5. Simulate a partial fill and a lost update, then reconcile the missing evidence.
6. Disconnect the dashboard while ticks and risk checks continue.
7. Replace the feed owner and restore subscriptions without forgetting positions.
8. Trigger one strategy's exit alert and preserve the other strategy's allocation.
9. Kill a worker after uncertain submission and recover without duplicate quantity.
10. Restore the ledger into an isolated environment and compare all accounting totals.

Evidence to deliver

Include a context diagram, a deployment diagram, one transaction sequence diagram, an order state machine, a threat model, a test report, an incident report, a restore report, and a dated cost model. Show the exact software versions and which cloud steps you executed. Label unsupported assumptions instead of hiding them.

Original Kubernetes review questions

Why can a Service exist without ready endpoints? Because its selectors may match nothing or its Pods may be unready. Why can a Pod be Pending? Scheduling, quota, resources, image/storage preparation, and policy can be involved; events narrow the cause. Why does a failed liveness probe differ from a failed readiness probe? One can cause restart; the other controls traffic eligibility. Why is a Secret not safe just because it is base64? Encoding is reversible and access policy still matters.

Why does HPA not solve a single broker's fixed quota? More workers can exceed the same external budget. Why is a StatefulSet not a database backup? Stable identity and volumes do not preserve an independent recoverable history. Why must an execution worker handle SIGTERM carefully? Stopping the local process does not cancel an in-flight external side effect. Why is `kubectl apply` not an acceptance test? It confirms object submission, not end-to-end business correctness.

Certification boundary

These labs build practical evidence relevant to Google Cloud engineering, architecture, operations, and security. They do not replace every objective in the current exam guide. Kubernetes-specific certifications have their own current task scope and exam environment; verify those official guides separately if you choose that path. Keep the goal concrete: explain, implement, test, deploy, break, recover, and defend your design.

Appendix A Glossary

Acceptance: A broker acknowledges an order request; it does not prove a trade.

Atomicity: A group of local database changes commits together or not at all.

Backpressure: Slowing or rejecting producers when downstream capacity is bounded.

Circuit breaker: A client-side failure-control pattern that temporarily stops calls after repeated failure. This is different from exchange circuit-price limits.

Correlation ID: An identifier linking related operations; it is not automatically an idempotency guarantee.

Durability: A stated level of persistence after acknowledgment, dependent on configuration and failure assumptions.

Fencing: Rejecting operations from an older owner using a monotonically newer ownership version, where the receiving system supports it.

Freshness: Whether data is recent enough for a particular use, considering its source and timestamps.

Idempotency: Repeating the same logical operation does not create another business effect.

Invariant: A rule that must always remain true, such as nonnegative remaining quantity.

Liveness: Evidence that a process can continue operating, distinct from readiness for a business action.

OLTP: Transaction-oriented storage for operational state changes.

OLAP: Analytical processing across historical or aggregated data.

Outbox: Durable events committed with the business changes they describe.

RPO: The acceptable data-loss window after a failure.

RTO: The target time to restore a service after a failure.

Reconciliation: Comparing local state with external authoritative evidence and resolving differences.

SLI and SLO: A measured service indicator and its target objective.

TOTP: A time-based code derived from a shared secret; one authentication factor, not broker authorization.

Trace: A linked view of an operation across components.

Appendix B Reference reading order

Use official documentation as a reference while building, not as a reason to postpone coding indefinitely. Begin with [Python](#), then [FastAPI](#), [pytest](#), and [Playwright Python](#). Read PostgreSQL transaction and indexing documentation when you reach durable state. Read current OWASP guidance before implementing public authentication.

For brokers, use the [DhanHQ API documentation](#) and [Kite Connect API documentation](#), together with the exact installed SDK source. Do not assume SDK documentation with a different version number matches your pinned package.

For cloud study, follow the current landing page and linked guide for your chosen certification. Product names and weights change. Keep a dated objective checklist and explicitly identify topics not exercised by the trading project. Official sample questions teach format; they do not guarantee your exam result.

Appendix C Complete paper lab source walkthrough

These listings let you study a complete small vertical slice without switching files. They are the same paper-only reference used by the accompanying tests. They do not connect to a broker, authenticate public users, or provide production durability. Type your own implementation first, then compare contracts and tests.

The domain module

Read this module first. It has no FastAPI, Redis, SQLite, or broker import. Its inputs are explicit, making money and risk behavior testable in isolation. RiskState records the previous protective state; advance returns a new state and an optional trigger. A trigger is only simulated as an immediate fill by the local API, not by a real broker adapter.

PYTHON

```
from dataclasses import dataclass
from decimal import Decimal, ROUND_CEILING, ROUND_FLOOR

def positive(value: Decimal) -> Decimal:
    if not value.is_finite() or value <= 0:
        raise ValueError("Expected a finite positive number")
    return value

def strict_quantity(capital: Decimal, price: Decimal) -> int:
    positive(price)
    if not capital.is_finite() or capital < 0:
        raise ValueError("Invalid capital")
    return int(capital // price)

def align_price(price: Decimal, tick: Decimal, side: str) -> Decimal:
    positive(price)
    positive(tick)
    if side not in {"BUY", "SELL"}:
        raise ValueError("Invalid side")
    rounding = ROUND_CEILING if side == "BUY" else ROUND_FLOOR
    result = (price / tick).to_integral_value(rounding=rounding) * tick
    return positive(result)

@dataclass(frozen=True)
class RiskState:
    side: str
    entry: Decimal
    target: Decimal
    stop: Decimal
    high_water: Decimal
    low_water: Decimal
    initial_stop_pct: Decimal
    trailing_enabled: bool
    trailing_pct: Decimal
    cost_enabled: bool
    cost_rr: Decimal

def advance(state: RiskState, price: Decimal) -> tuple[RiskState, str | None]:
    positive(price)
    if state.side not in {"BUY", "SELL"}:
        raise ValueError("Invalid side")
    high = max(state.high_water, price)
    low = min(state.low_water, price)
    stop = state.stop
    hundred = Decimal("100")
    if state.side == "BUY":
```

```

        if state.trailing_enabled:
            stop = max(stop, high * (1 - state.trailing_pct / hundred))
        if state.cost_enabled and high >= state.entry * (
            1 + state.initial_stop_pct * state.cost_rr / hundred
        ):
            stop = max(stop, state.entry)
        reason = "TARGET" if price >= state.target else "STOP" if price <= stop else None
    else:
        if state.trailing_enabled:
            stop = min(stop, low * (1 + state.trailing_pct / hundred))
        if state.cost_enabled and low <= state.entry * (
            1 - state.initial_stop_pct * state.cost_rr / hundred
        ):
            stop = min(stop, state.entry)
        reason = "TARGET" if price <= state.target else "STOP" if price >= stop else None
    updated = RiskState(
        state.side, state.entry, state.target, stop, high, low,
        state.initial_stop_pct, state.trailing_enabled, state.trailing_pct,
        state.cost_enabled, state.cost_rr,
    )
    return updated, reason

def marked_pnl(side: str, entry: Decimal, price: Decimal, quantity: int) -> Decimal:
    if side not in {"BUY", "SELL"}:
        raise ValueError("Invalid side")
    if quantity <= 0:
        raise ValueError("Invalid quantity")
    positive(entry)
    positive(price)
    return (price - entry) * quantity * (1 if side == "BUY" else -1)

```

The API and SQLite transaction boundary

Read model validation, then transaction, then each route. Every database connection is scoped and closed. BEGIN IMMEDIATE serializes the small SQLite teaching workload. It is not a scalable replacement for designing PostgreSQL transactions and queues. The unique open-position index and durable receipt table demonstrate two different invariants: one open allocation per strategy/symbol, and one effect per event identity.

Configuration is copied into each new position, so later changes do not silently rewrite its entry-time rules. The input price is synthetic and caller-supplied only because this is a simulator. Never transfer that trust assumption to a live webhook.

PYTHON

```

"""Paper-only API. Bind to loopback; authentication is a later assignment."""
from contextlib import contextmanager
from decimal import Decimal
import hashlib
import json
import os
from pathlib import Path
import re
import sqlite3
from typing import Literal
import uuid

from fastapi import FastAPI, HTTPException
from fastapi.responses import HTMLResponse, JSONResponse
from pydantic import BaseModel, ConfigDict, Field, field_validator

from .domain import RiskState, advance, marked_pnl

class Strategy(BaseModel):
    model_config = ConfigDict(extra="forbid")
    name: str = Field(min_length=1, max_length=120)

```

```

side: Literal["BUY", "SELL"] = "BUY"
quantity: int = Field(default=1, strict=True, gt=0, le=10000)
target_pct: Decimal = Field(default=Decimal("2"), gt=0, lt=100, allow_inf_nan=False)
stop_pct: Decimal = Field(default=Decimal("1"), gt=0, lt=100, allow_inf_nan=False)
trailing_enabled: bool = False
trailing_pct: Decimal = Field(default=Decimal("1"), gt=0, lt=100, allow_inf_nan=False)
cost_enabled: bool = False
cost_rr: Decimal = Field(default=Decimal("2"), gt=0, le=100, allow_inf_nan=False)
retry_count: int = Field(default=0, strict=True, ge=0, le=3)

@field_validator("name")
@classmethod
def clean_name(cls, value):
    value = " ".join(value.split())
    if not value:
        raise ValueError("Strategy name is required")
    return value

class Signal(BaseModel):
    model_config = ConfigDict(extra="forbid")
    event_id: str = Field(min_length=1, max_length=100)
    strategy: str = Field(min_length=1, max_length=120)
    symbol: str = Field(min_length=1, max_length=30)
    price: Decimal = Field(gt=0, le=Decimal("10000000"), allow_inf_nan=False)

    @field_validator("symbol")
    @classmethod
    def clean_symbol(cls, value):
        value = value.strip().upper()
        if not re.fullmatch(r"[A-Z0-9&.-]{1,30}", value):
            raise ValueError("Invalid paper symbol")
        return value

class Tick(BaseModel):
    model_config = ConfigDict(extra="forbid")
    symbol: str = Field(min_length=1, max_length=30)
    price: Decimal = Field(gt=0, le=Decimal("10000000"), allow_inf_nan=False)

def canonical(name):
    return " ".join(name.split()).casefold()

def create_app(database: str | Path) -> FastAPI:
    database = Path(database)
    database.parent.mkdir(parents=True, exist_ok=True)

    @contextmanager
    def transaction():
        connection = sqlite3.connect(database, timeout=5, isolation_level=None)
        connection.row_factory = sqlite3.Row
        try:
            connection.execute("BEGIN IMMEDIATE")
            yield connection
            connection.commit()
        except BaseException:
            connection.rollback()
            raise
        finally:
            connection.close()

    with transaction() as db:
        db.execute("CREATE TABLE IF NOT EXISTS configs (name TEXT PRIMARY KEY, payload TEXT NOT NULL)")
        db.execute("CREATE TABLE IF NOT EXISTS receipts (event_id TEXT PRIMARY KEY, digest TEXT NOT NULL, position_id TEXT NOT NULL)")
        db.execute("""CREATE TABLE IF NOT EXISTS positions (
            id TEXT PRIMARY KEY, strategy TEXT NOT NULL, symbol TEXT NOT NULL,
            status TEXT NOT NULL, payload TEXT NOT NULL)""")

```

```

db.execute("""CREATE UNIQUE INDEX IF NOT EXISTS one_open_position
ON positions(strategy, symbol) WHERE status = 'OPEN'""")

app = FastAPI(title="Trading School Paper Lab")

@app.exception_handler(HTTPException)
async def http_error(request, exc):
    return JSONResponse(status_code=exc.status_code, content={"ok": False, "detail": str(exc.detail)})

@app.get("/health")
def health():
    return {"ok": True, "mode": "PAPER_ONLY"}

@app.get("/", response_class=HTMLResponse)
def home():
    return Path(__file__).with_name("dashboard.html").read_text(encoding="utf-8")

@app.post("/api/configs")
def save_strategy(config: Strategy):
    data = config.model_dump(mode="json")
    with transaction() as db:
        db.execute("""INSERT INTO configs(name, payload) VALUES (?, ?)
ON CONFLICT(name) DO UPDATE SET payload=excluded.payload""",
        (canonical(config.name), json.dumps(data)))
    return {"ok": True, "strategy": data}

@app.get("/api/configs")
def configs():
    with transaction() as db:
        rows = db.execute("SELECT payload FROM configs ORDER BY name").fetchall()
    return {"ok": True, "strategies": [json.loads(row[0]) for row in rows]}

@app.post("/api/signals")
def signal(payload: Signal):
    name = canonical(payload.strategy)
    normalized = payload.model_dump(mode="json")
    normalized["strategy"] = name
    normalized["price"] = str(payload.price.normalize())
    digest = hashlib.sha256(json.dumps(normalized, sort_keys=True).encode()).hexdigest()
    with transaction() as db:
        previous = db.execute("SELECT * FROM receipts WHERE event_id=?", (payload.event_id,)).fetchone()
        if previous:
            if previous["digest"] != digest:
                raise HTTPException(409, "EVENT_ID_CONFLICT")
            return {"ok": True, "duplicate": True, "position_id": previous["position_id"]}
        row = db.execute("SELECT payload FROM configs WHERE name=?", (name,)).fetchone()
        if not row:
            raise HTTPException(404, "CONFIG_NOT_FOUND")
        config = Strategy.model_validate_json(row[0])
        existing = db.execute("SELECT id FROM positions WHERE strategy=? AND symbol=? AND status='OPEN'",
        (name, payload.symbol)).fetchone()
        if existing:
            raise HTTPException(409, "ALREADY_OPEN")
        sign = Decimal("1") if config.side == "BUY" else Decimal("-1")
        position_id = uuid.uuid4().hex
        # The local simulator fills immediately at the supplied synthetic price.
        # A real adapter must wait for independent broker fill evidence.
        position = {
            "id": position_id, "strategy": config.name, "symbol": payload.symbol,
            "side": config.side, "quantity": config.quantity,
            "entry": str(payload.price), "ltp": str(payload.price),
            "target": str(payload.price * (1 + sign * config.target_pct / 100)),
            "stop": str(payload.price * (1 - sign * config.stop_pct / 100)),
            "high_water": str(payload.price), "low_water": str(payload.price),
            "config": config.model_dump(mode="json"), "pn1": "0",
            "status": "OPEN", "exit_reason": None,
        }
    db.execute("INSERT INTO positions VALUES (?, ?, ?, ?, ?)",
    (position_id, name, payload.symbol, "OPEN", json.dumps(position)))
    db.execute("INSERT INTO receipts VALUES (?, ?, ?)", (payload.event_id, digest, position_id))

```

```

        return {"ok": True, "duplicate": False, "position_id": position_id}

@app.post("/api/ticks")
def tick(payload: Tick):
    changed = []
    with transaction() as db:
        rows = db.execute("SELECT payload FROM positions WHERE symbol=? AND status='OPEN'",
                           (payload.symbol.strip().upper(),)).fetchall()
        for row in rows:
            position = json.loads(row[0])
            config = Strategy.model_validate(position["config"])
            state = RiskState(
                position["side"], Decimal(position["entry"]), Decimal(position["target"]),
                Decimal(position["stop"]), Decimal(position["high_water"]), Decimal(position["low_water"]),
                config.stop_pct, config.trailing_enabled, config.trailing_pct,
                config.cost_enabled, config.cost_rr,
            )
            updated, reason = advance(state, payload.price)
            position.update(
                ltp=str(payload.price), stop=str(updated.stop),
                high_water=str(updated.high_water), low_water=str(updated.low_water),
                pnl=str(marked_pnl(state.side, state.entry, payload.price, position["quantity"])),
            )
            if reason:
                position.update(status="CLOSED", exit_reason=reason, exit_price=str(payload.price))
            db.execute("UPDATE positions SET status=?, payload=? WHERE id=?",
                       (position["status"], json.dumps(position), position["id"]))
            changed.append(position)
    return {"ok": True, "positions": changed}

@app.get("/api/positions")
def positions():
    with transaction() as db:
        rows = db.execute("SELECT payload FROM positions ORDER BY rowid").fetchall()
    return {"ok": True, "positions": [json.loads(row[0]) for row in rows]}

return app

def default_app():
    return create_app(os.environ.get("PAPERLAB_DB", "paperlab.sqlite3"))

```

The browser integration test

This test launches its own loopback server against a temporary database, interacts with the real form, sends synthetic events through the API, and observes the resulting table. Notice cleanup in `finally`, readiness waiting with a deadline, no broker secret, and separate mobile/desktop runs. The final test deliberately returns HTML where the browser expected JSON.

PYTHON

```

"""Real browser against a disposable server and a synthetic paper broker."""
import os
from pathlib import Path
import socket
import subprocess
import sys
import time
import uuid

import httpx
import pytest
from playwright.sync_api import expect, sync_playwright

@pytest.fixture(scope="module")
def server(tmp_path_factory):
    temp = tmp_path_factory.mktemp("paper-browser")

```

```

with socket.socket() as sock:
    sock.bind(("127.0.0.1", 0))
    port = sock.getsockname()[1]
base = f"http://127.0.0.1:{port}"
env = {**os.environ, "PAPERLAB_DB": str(temp / "paper.sqlite3")}
root = Path(__file__).resolve().parents[1]
with (temp / "server.log").open("w", encoding="utf-8") as output:
    process = subprocess.Popen([
        sys.executable, "-m", "uvicorn", "paperlab.api:default_app", "--factory",
        "--host", "127.0.0.1", "--port", str(port),
    ], cwd=root, env=env, stdout=output, stderr=subprocess.STDOUT)
try:
    deadline = time.monotonic() + 20
    while time.monotonic() < deadline:
        if process.poll() is not None:
            raise RuntimeError("Paper test server exited; inspect its temporary log")
        try:
            if httpx.get(base + "/health", timeout=1).status_code == 200:
                break
        except httpx.HTTPError:
            pass
        time.sleep(0.1)
    else:
        raise RuntimeError("Paper test server did not become ready")
yield base
finally:
    process.terminate()
    try:
        process.wait(timeout=10)
    except subprocess.TimeoutExpired:
        process.kill()
        process.wait(timeout=5)

@pytest.mark.parametrize("viewport", [{"width": 1440, "height": 1000}, {"width": 390, "height": 844}])
def test_form_to_backend_to_closed_position(server, viewport):
    with sync_playwright() as playwright:
        try:
            browser = playwright.chromium.launch(channel="chrome", headless=True)
        except Exception:
            browser = playwright.chromium.launch(headless=True)
        context = browser.new_context(viewport=viewport)
        page = context.new_page()
        errors = []
        page.on("pageerror", lambda error: errors.append(str(error)))
        try:
            page.goto(server)
            name = "Browser " + uuid.uuid4().hex[:8]
            symbol = "DEMO" + str(viewport["width"])
            page.get_by_label("Strategy name", exact=True).fill(name)
            page.get_by_role("button", name="Save strategy", exact=True).click()
            expect(page.get_by_role("status")).to_have_text("Saved " + name)
            with httpx.Client(base_url=server) as client:
                configs = client.get("/api/configs").json()["strategies"]
                assert next(item for item in configs if item["name"] == name)["retry_count"] == 0
                response = client.post("/api/signals", json={
                    "event_id": uuid.uuid4().hex, "strategy": name, "symbol": symbol, "price": "100",
                })
                assert response.status_code == 200
            page.get_by_role("button", name="Refresh positions").click()
            row = page.get_by_role("row").filter(has=page.get_by_role("cell", name=symbol, exact=True))
            expect(row).to_contain_text("OPEN")
            assert client.post("/api/ticks", json={"symbol": symbol, "price": "103"}).status_code == 200
            page.get_by_role("button", name="Refresh positions").click()
            expect(row).to_contain_text("CLOSED")
            expect(row).to_contain_text("TARGET")
            assert page.evaluate("document.documentElement.scrollHeight <= innerWidth")
            assert not errors
        finally:
            context.close()

```

```

        browser.close()

def test_html_gateway_failure_is_readable(server):
    with sync_playwright() as playwright:
        try:
            browser = playwright.chromium.launch(channel="chrome", headless=True)
        except Exception:
            browser = playwright.chromium.launch(headless=True)
        page = browser.new_page()
        try:
            page.route("**/api/positions", lambda route: route.fulfill(
                status=502, content_type="text/html", body="<h1>Bad Gateway</h1>"))
            page.goto(server)
            expect(page.get_by_role("status")).to_have_text("Unexpected server response (502)")
        finally:
            browser.close()

```

Exercises after reading the listings

Add strategy enabled/disabled without changing existing persisted positions. Add an API test and browser test for it. Then separate acceptance from simulated fill so `/api/signals` no longer immediately creates a position. Introduce a paper-order ledger and an explicit synthetic execution endpoint. Deliver duplicate fill events and verify quantity is unchanged. Finally replace SQLite with a repository interface and implement PostgreSQL without changing your pure domain rules.